MAJOR PROJECT REPORT

Design and Implementation of a Secure Azure Cloud Environment

Prepared by:

ABDUL HAMEED I

CS-T8/May-10355

Date: July 10, 2026

Project Objectives

This report presents the findings of a security assessment conducted on the Microsoft Azure environment deployed for XYZ Solutions within the resource group interns-rg. The assessment covers a Linux virtual machine, an Azure Storage account, and supporting identity, monitoring, and threat-detection services.

The objective of the engagement was to establish baseline preventive security controls consistent with the Shared Responsibility Model and the Principle of Least Privilege, and to validate those controls through hands-on configuration, testing, and evidence collection. Controls implemented include SSH port hardening, role-based access control (RBAC) with segregation of duties, private blob storage with Shared Access Signature (SAS) delegation, centralized monitoring through Azure Monitor, threat detection through Microsoft Sentinel, Just-In-Time (JIT) VM access, and an external vulnerability scan using Nmap.

Overall, the environment moved from a default, broadly-exposed configuration to a hardened baseline in which administrative ports are closed by default, storage is private by default, and security-relevant events are centrally logged and monitored. Residual risks and recommended next steps are detailed in Sections 2 and 4.

Task 1: Secure Virtual Machine Deployment & Hardening

A Linux-based Virtual Machine was provisioned. To reduce exposure to automated scanning tools, standard SSH access via port 22 was hardened by reconfiguring the server to use port 2244 and creating custom firewall rules.

Parameter	Configuration Detail
VM Name	linux
Resource Group	interns-rg
OS Image	SLES 15 SP5 (Initial) / Ubuntu 24.04 (Final)
IP Address	20.244.6.120
Hardened SSH Port	2244

Step-by-step Execution and Evidence:

1. Virtual Machine was provisioned successfully in the Portal. The state was checked and confirmed as Running.

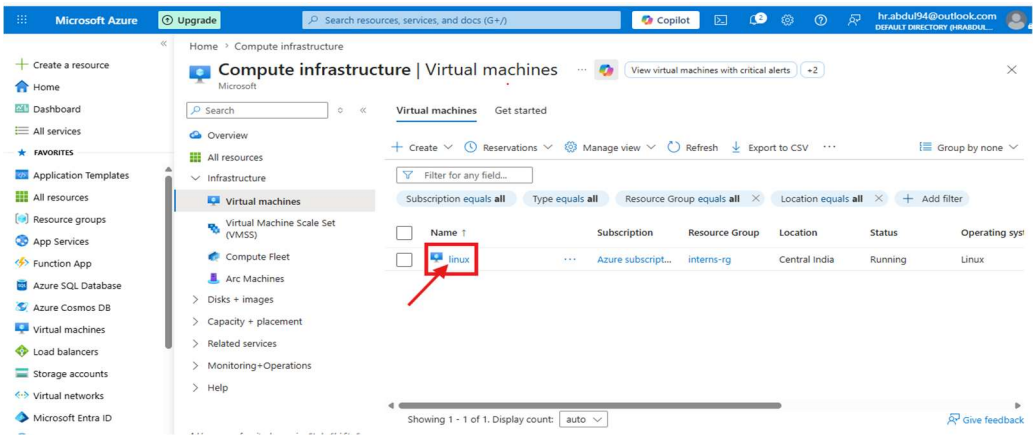

Figure 1: Azure Portal list showing active 'linux' VM in Running state.

2. System properties, IP specification, size details, and billing models were verified on the VM essentials tab.

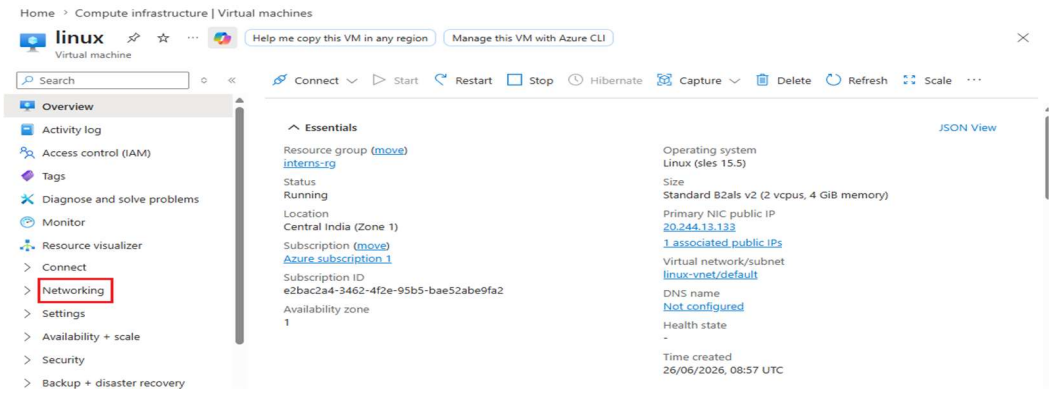

Figure 2: Essentials panel displaying details of the VM.

3. An initial baseline test confirmed port 22 was open and responsive to administrative connections.

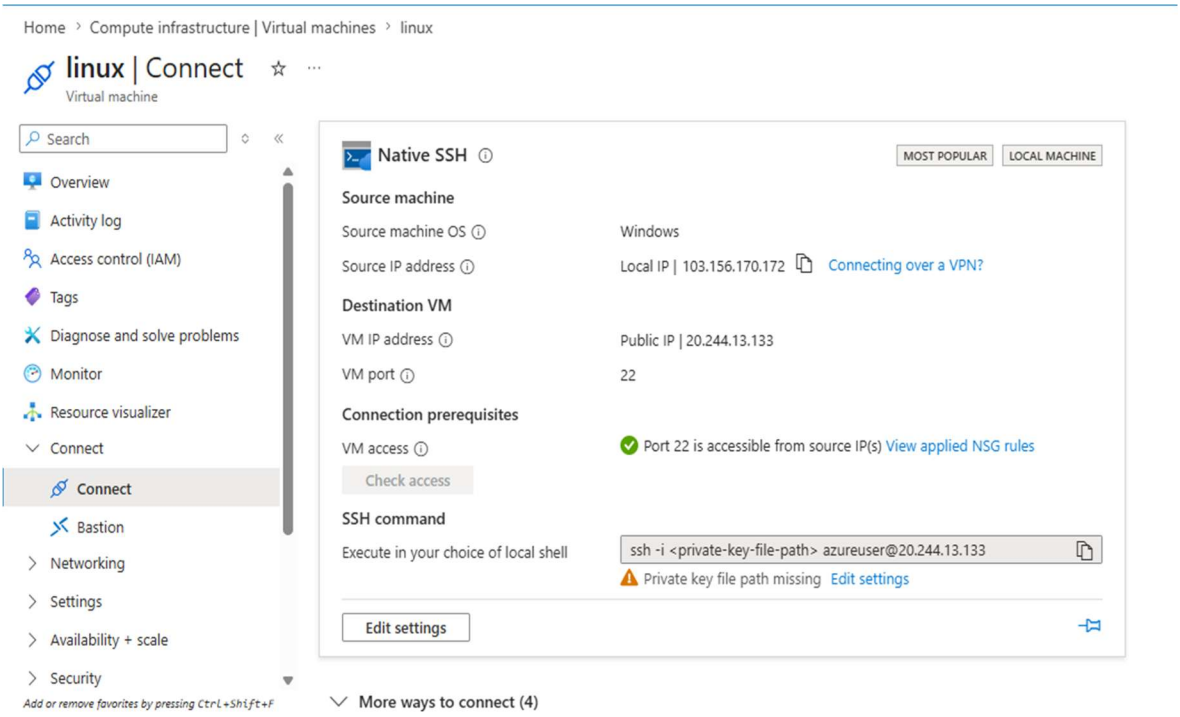

Figure 3: Connection diagnostic confirming standard port 22 is reachable.

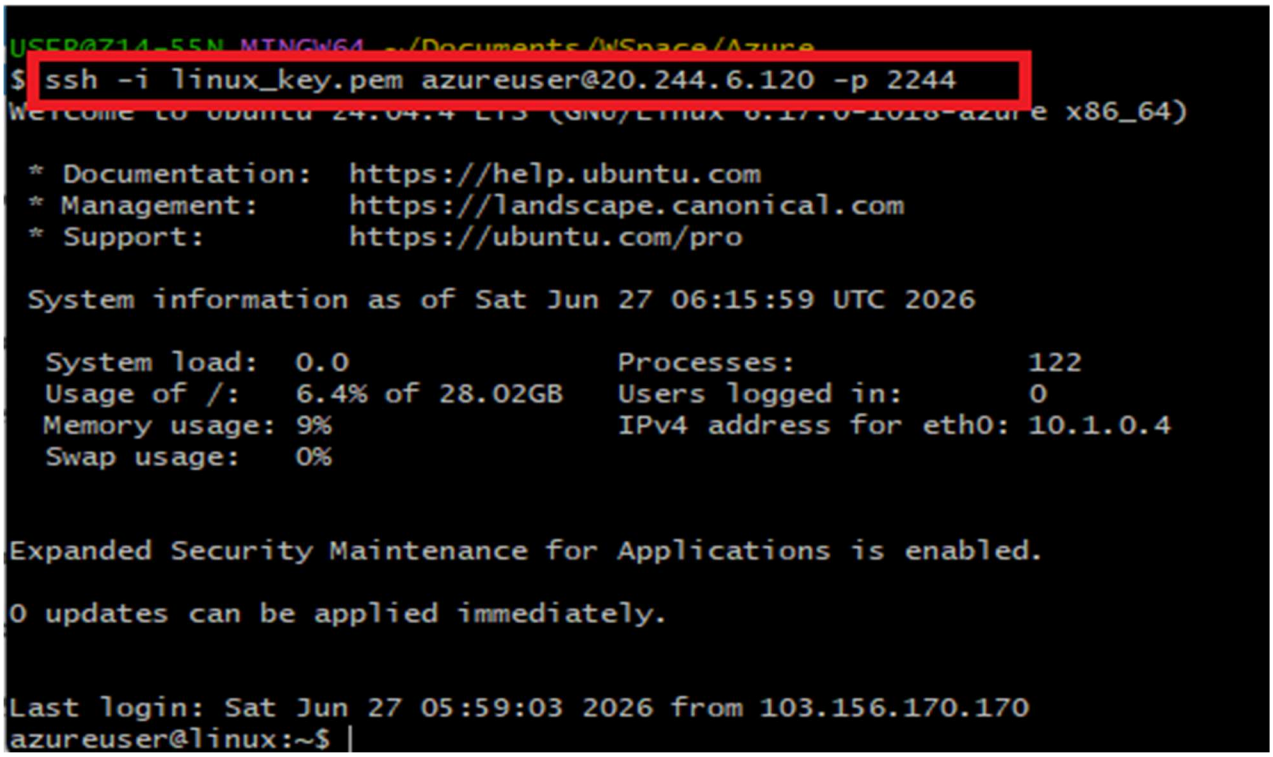

Figure 4: Terminal console displaying successful baseline login on default port 22.

4. The host server configurations were hardened. Port 22 was blocked, and a custom rule allowed traffic on port 2244.

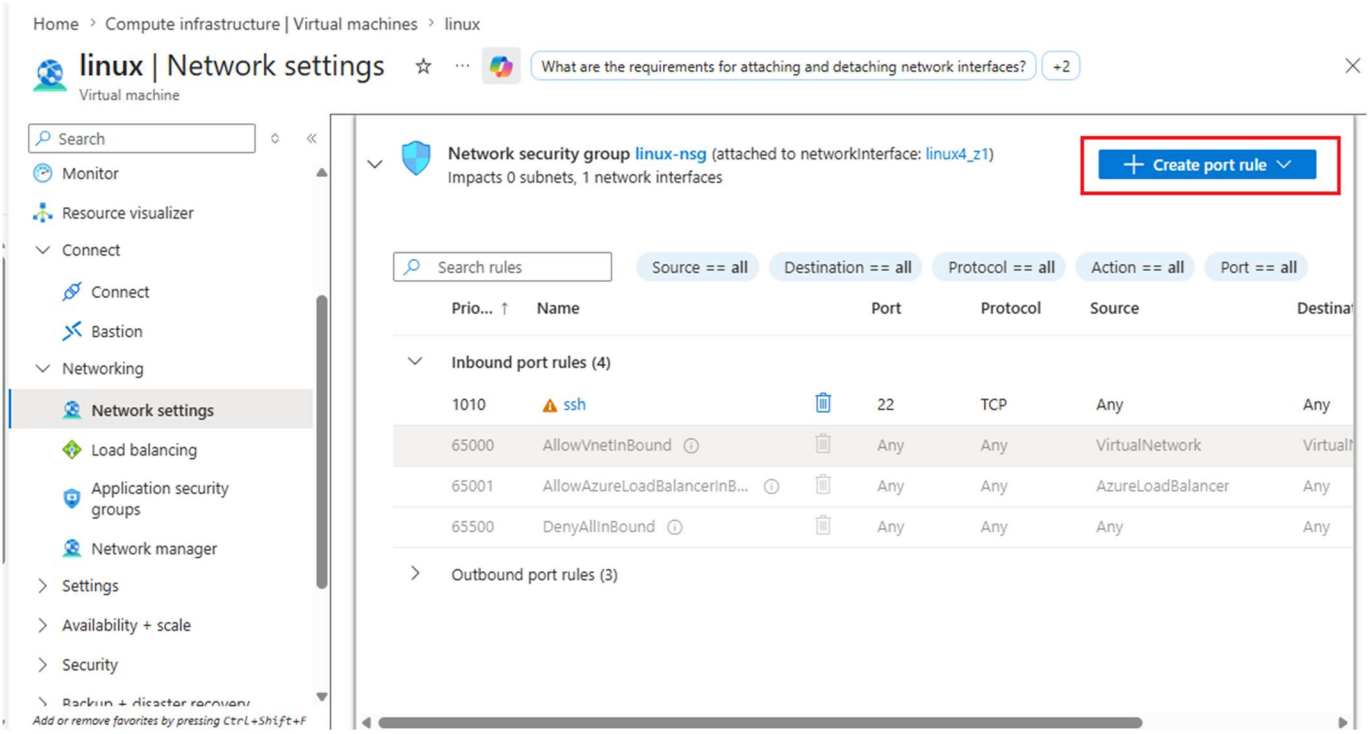

Figure 5: Azure Portal - Defining parameters to open Custom SSH Port 2244.

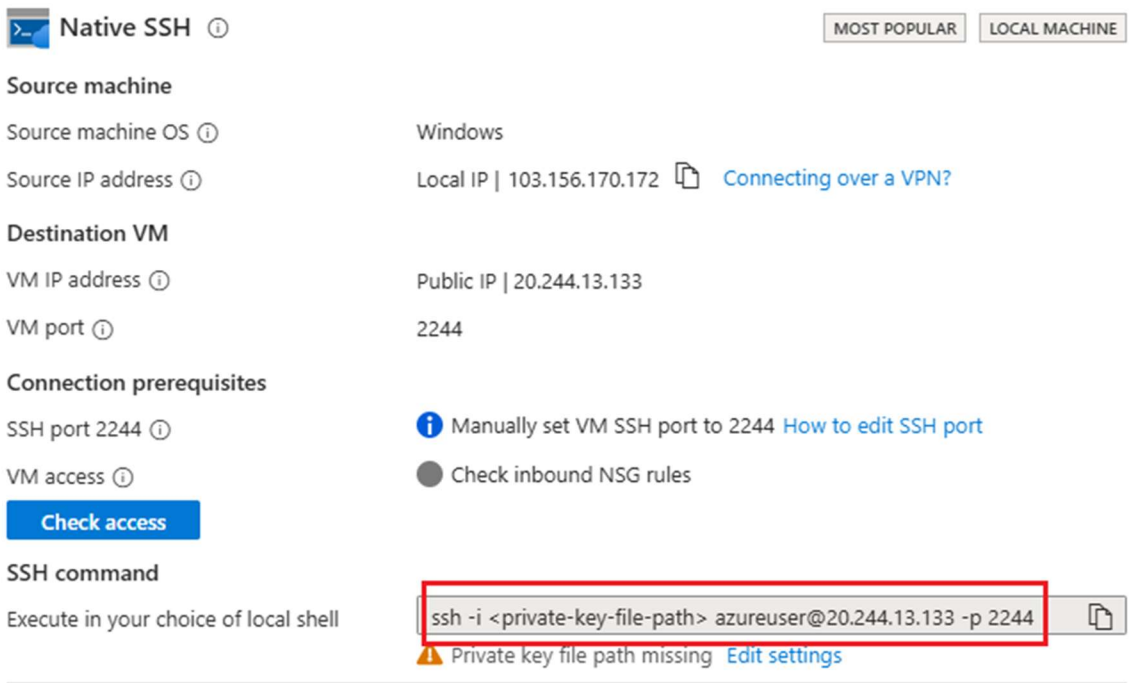

Figure 6: NSG inbound rule lists showing Rule 1009 (Port 2244) active and Port 22 default rules removed.

5. Hardening was verified. Standard port 22 connections were refused, and successful connections were logged on custom port 2244.

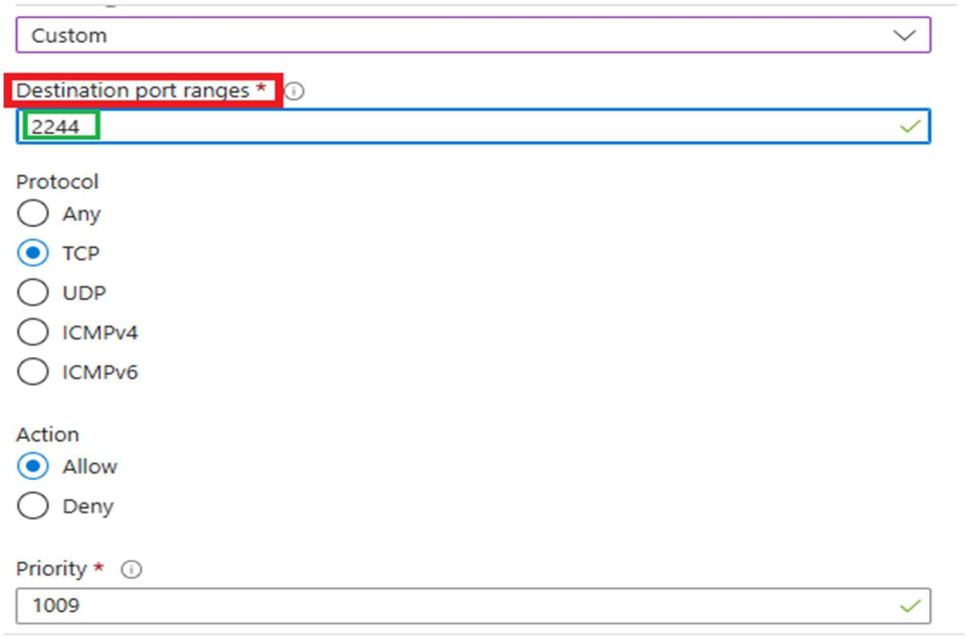

Figure 7: Terminal showing 'Connection refused' error on port 22 indicating successful blocking.

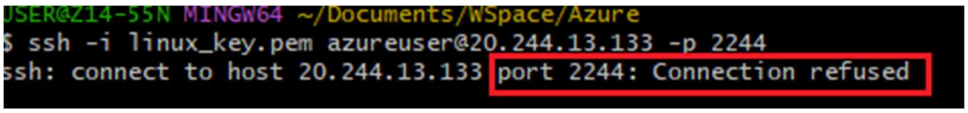

Figure 8: Port 2244 network path check showing successful connection routing.

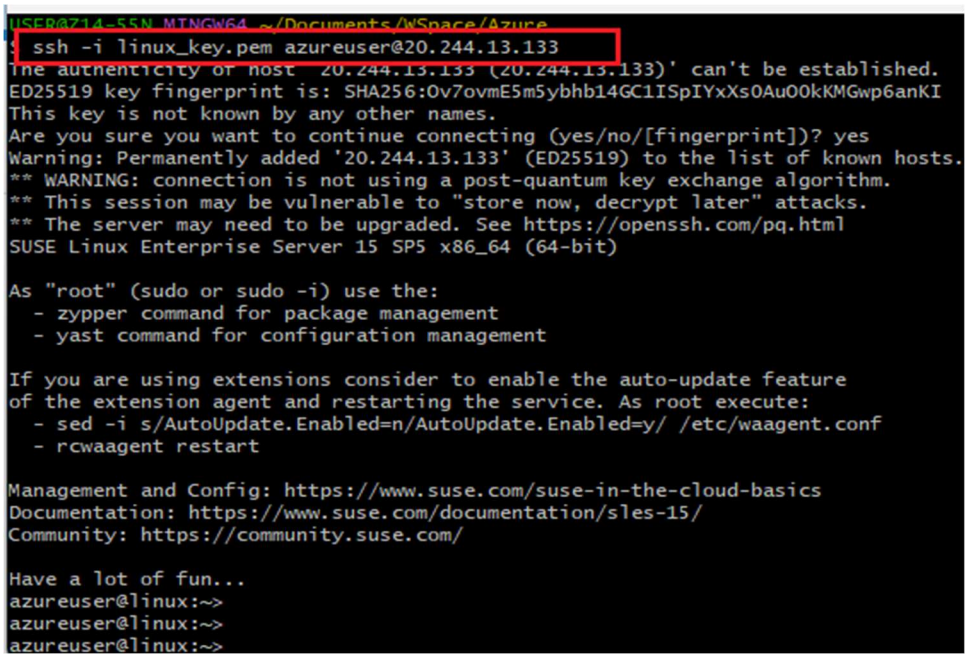

Figure 9: Terminal SSH session successfully established on custom port 2244.

Task 2: Identity & Access Management (IAM) and RBAC

Role-Based Access Control (RBAC) was implemented at the resource group level to enforce segregation of duties. Two separate administrators were created, ensuring infrastructure managers could not access storage systems, and vice versa.

Identity Username	Assigned RBAC Roles	Authorized Boundaries
infrauser	Virtual Machine Contributor, Network Contributor	Manage VMs, networks, interfaces. Denied Storage data access.
storageuser	Storage Account Contributor	Manage Storage accounts, containers, blobs. Denied VM operations.

Step-by-step Execution and Evidence:

1. The user identities were created in Microsoft Entra ID Directory.

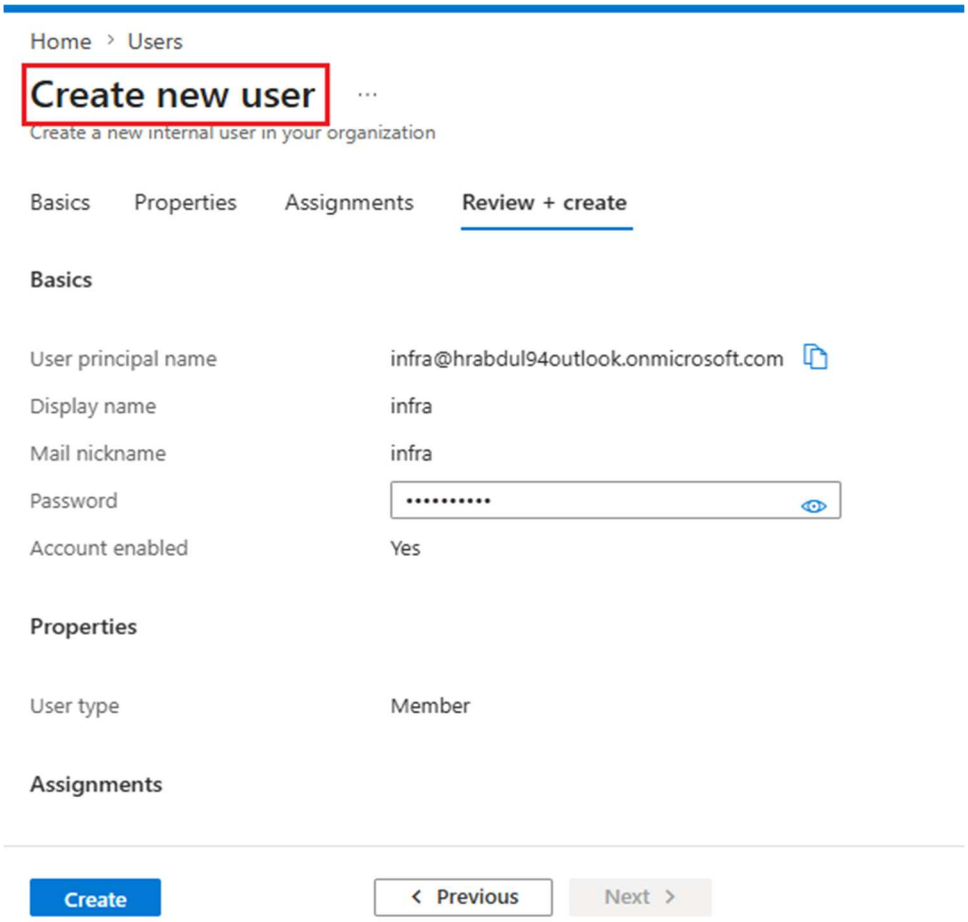

Figure 10: Microsoft Entra ID - Active user accounts listing 'infrauser' and 'storageuser'.

2. Permissions were granted by mapping corresponding RBAC roles at the Resource Group scope.

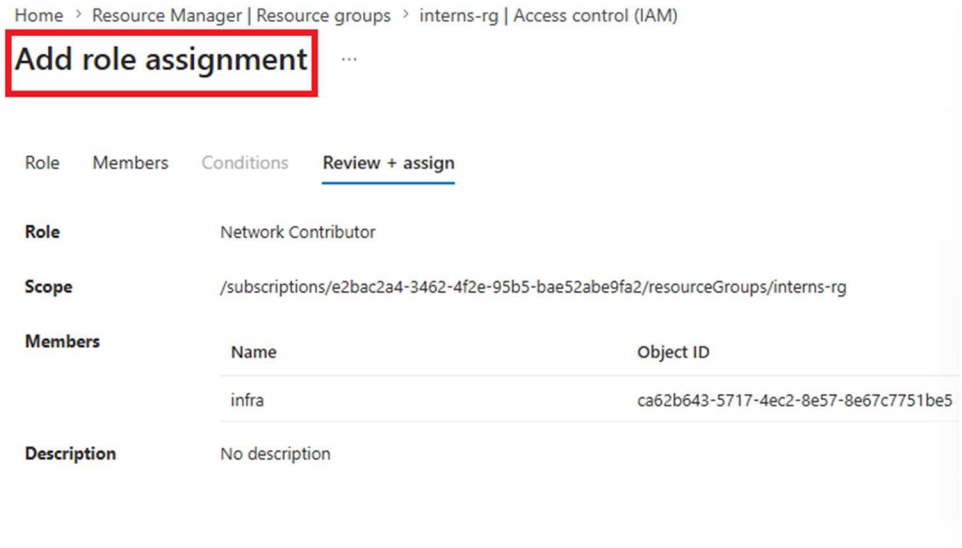

Figure 11: Assignment of VM Contributor permissions to 'infrauser'.

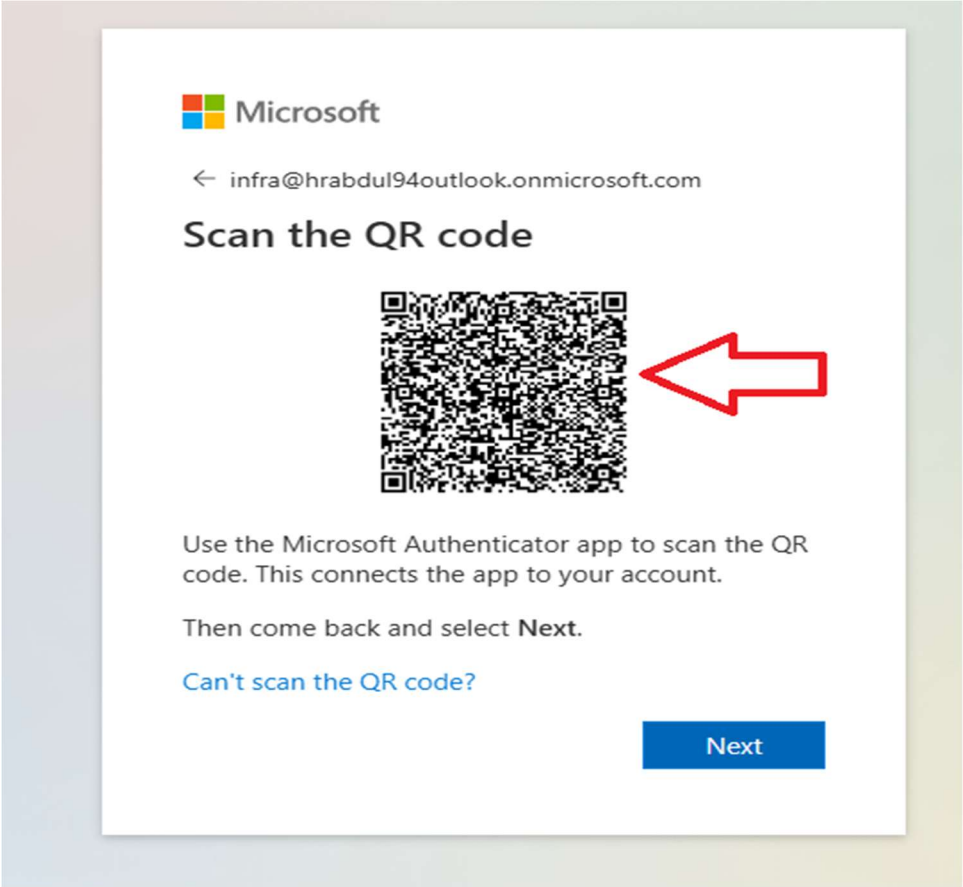

Figure 12: Assignment of Storage Account Contributor permissions to 'storageuser'.

3. Cross-resource restrictions were validated by logging into each user account.

Figure 13: Logging in as 'storageuser' - User session initialization screen (HEIF source validation).

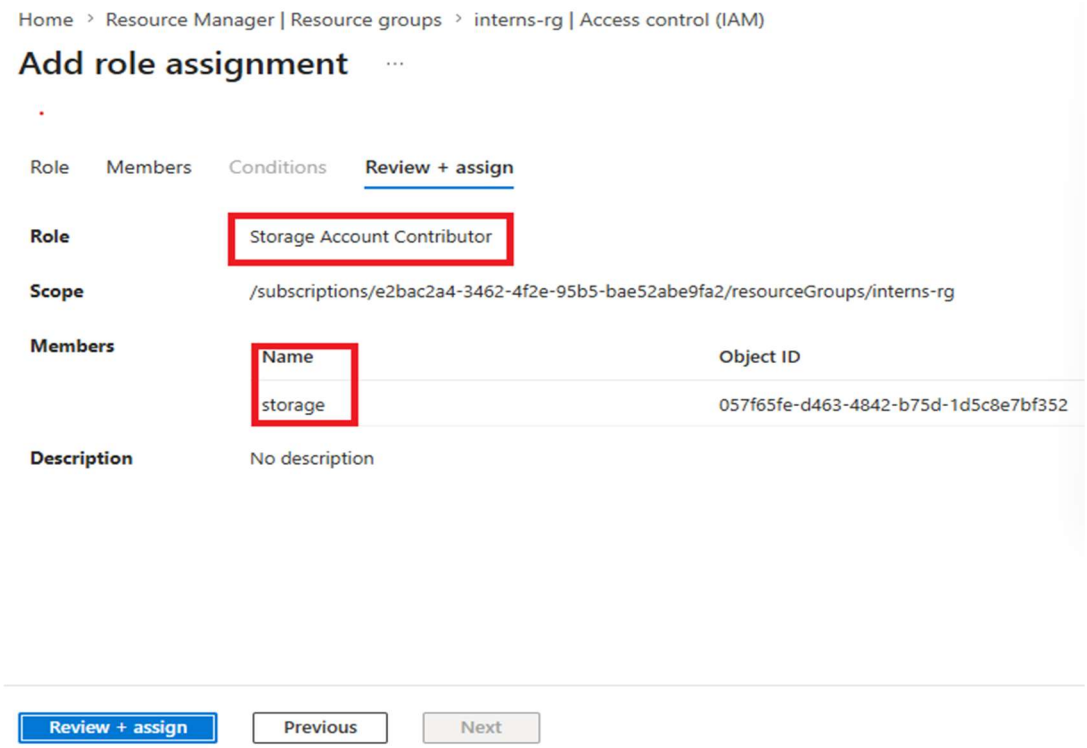

Figure 14: Storage admin validation showing full access to storage resources under 'storageuser'.

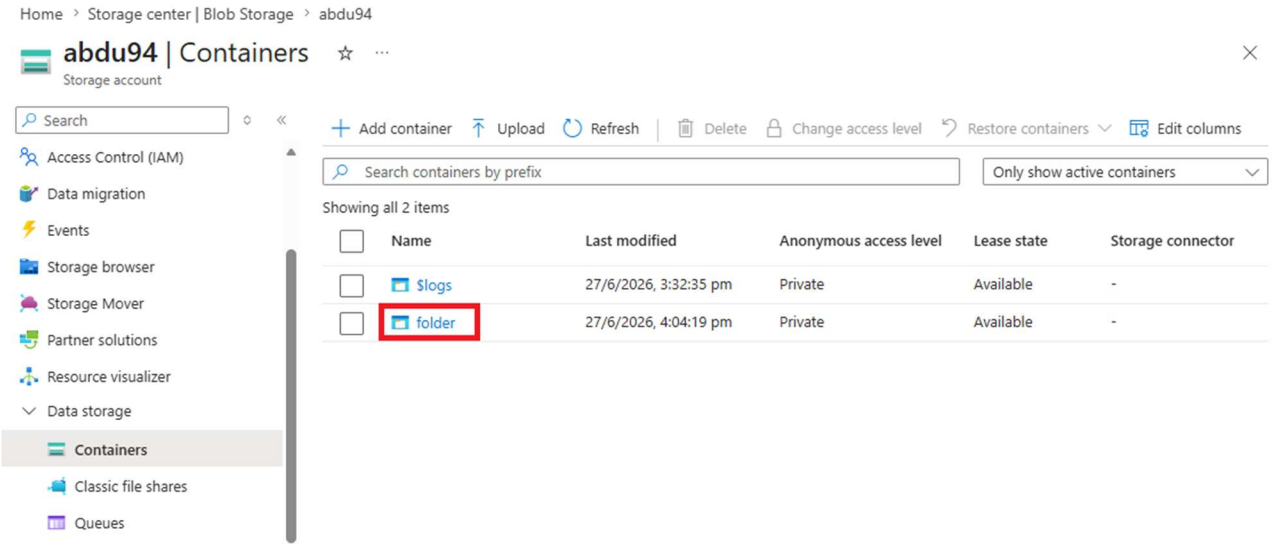

Figure 15: Validating resource lists, ensuring no compute assets are manageable by 'storageuser'.

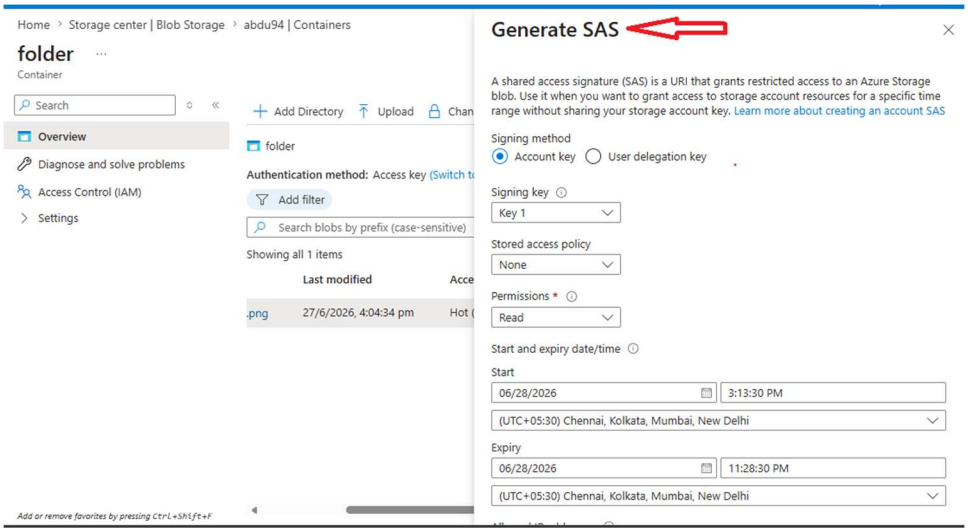

Figure 16: Verification of active RBAC session policies for storage actions.

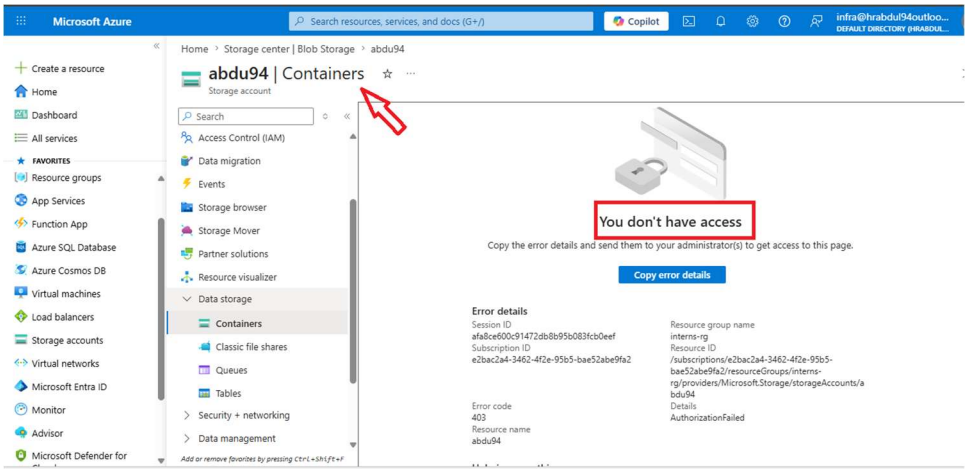

Figure 17: Logging in as 'infrauser' - Access Denied is shown when trying to view Storage account objects.

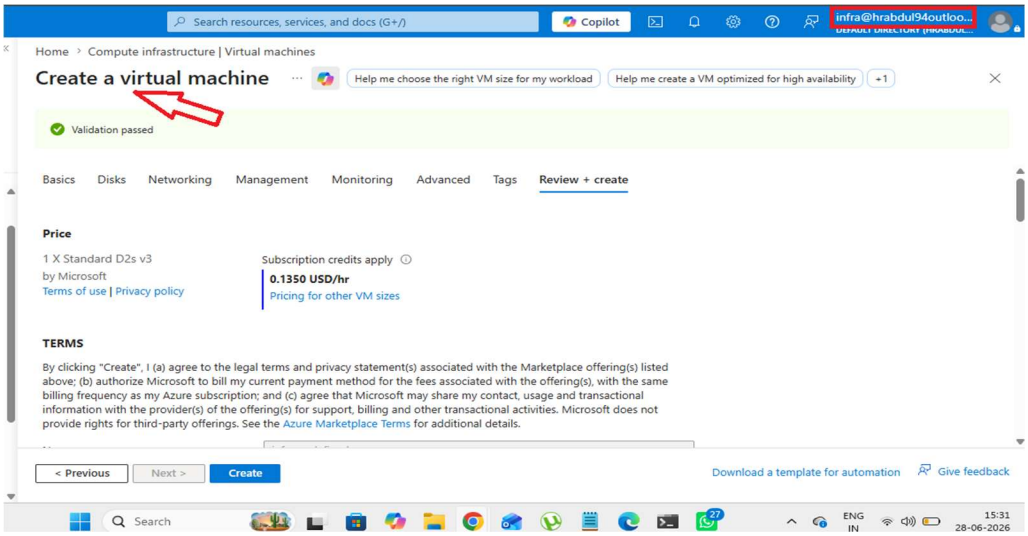

Figure 18: Logging in as 'storageuser' - Access Denied is shown when attempting to inspect or manage Virtual Machines.

Task 3: Secure Storage Account & SAS Access Configuration

A private container was created inside an Azure Storage Account to secure corporate files. Public anonymous access was disabled. Granular, time-bound access was configured using Shared Access Signatures (SAS).

Parameter	Configuration Detail
Storage Account Name	abdu94
Container Name	folder1
Access Level	Private (No anonymous public read access)
SAS Permission	Read-only
Signing Key	Key 1

Step-by-step Execution and Evidence:

1. Storage Account was created. A blob container named 'folder1' was configured with public access set to Private.

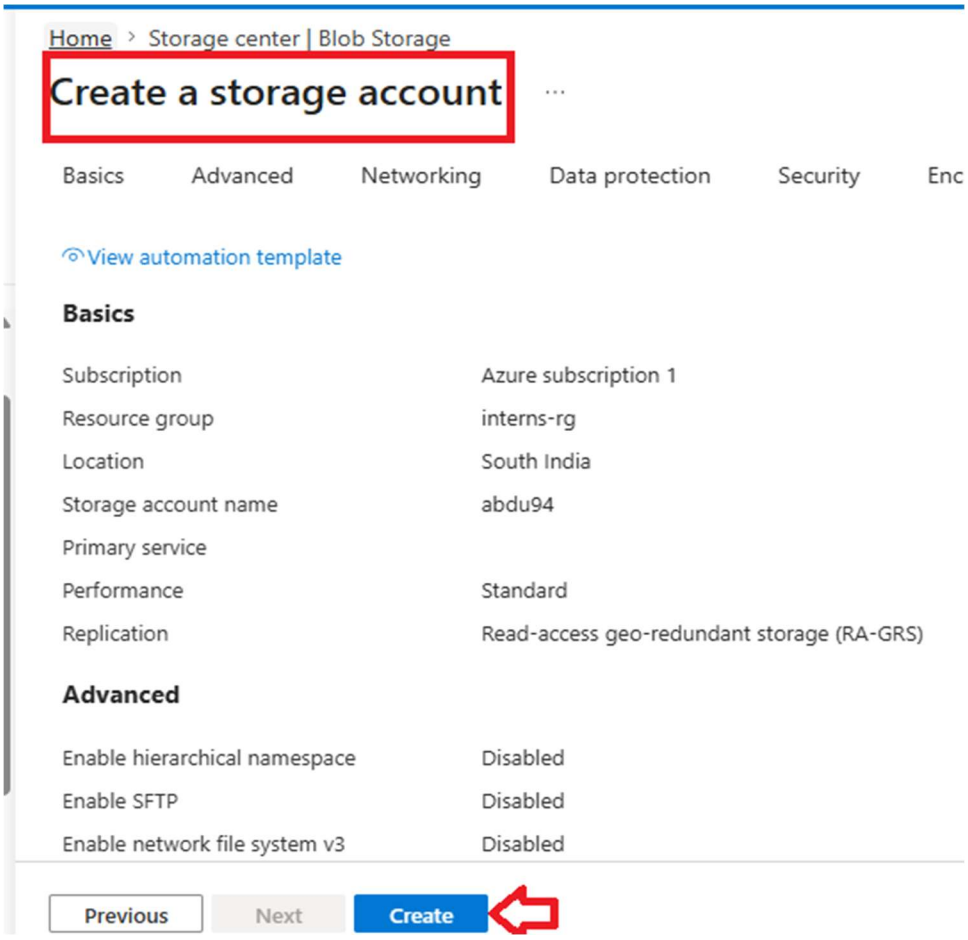

Figure 19: Essentials dashboard of Storage Account 'abdu94'.

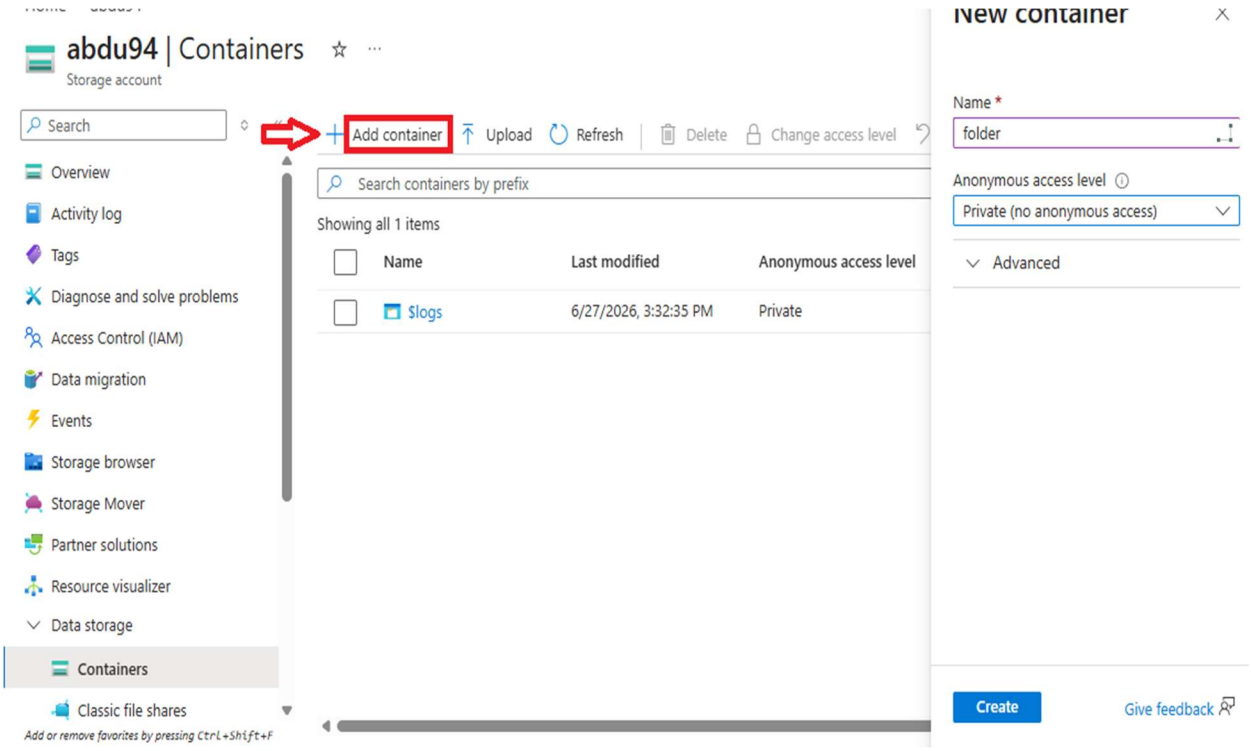

Figure 20: Creating container 'folder1' and checking its Private access properties.

2. A file named 'file.pdf' was uploaded to the container.

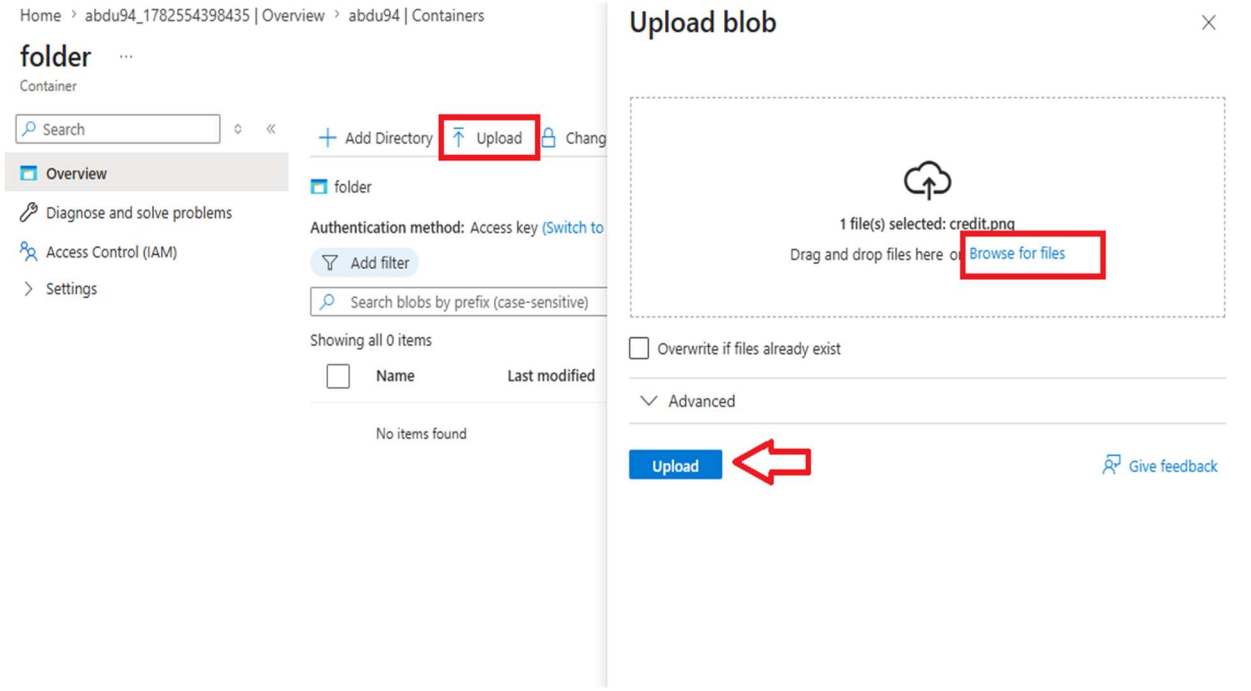

Figure 21: Container blob interface listing the uploaded file.

3. An anonymous read attempt was performed using the direct public URL. Access was denied, returning an XML error.

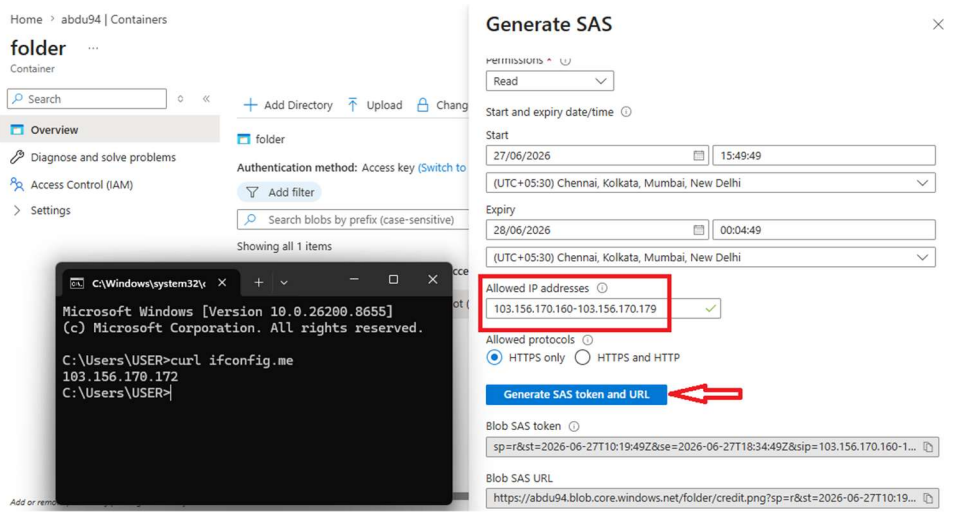

Figure 22: Public URL access attempt blocked, returning a ResourceNotFound/PublicAccessNotAllowed error.

4. A Shared Access Signature (SAS) token was generated with Read permissions and a specific time-bound expiration.

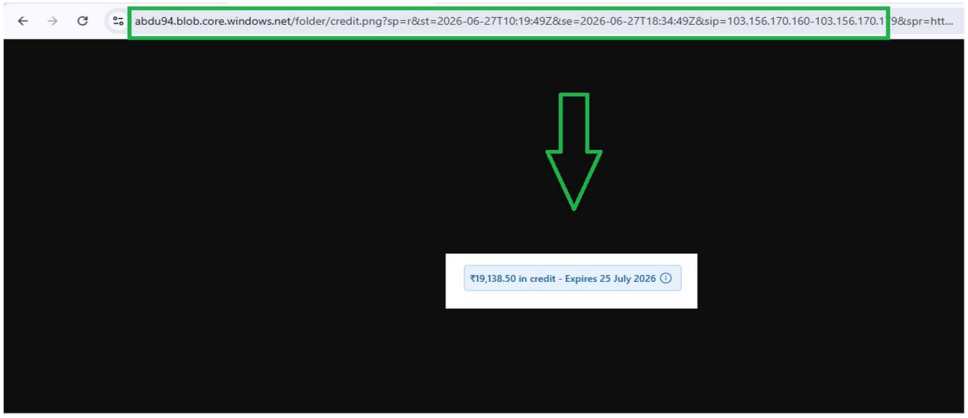

Figure 23: Generating blob SAS token specifying Read permission and custom start/end timestamps.

5. Access was validated using the SAS URL in a browser. The file successfully displayed, confirming authorized access.

Figure 24: Successful rendering of file.pdf when accessed via the generated SAS token URL.

Task 4: Azure Monitoring and Dashboard Visualization

A centralized monitoring solution was implemented using Azure Monitor, Log Analytics Workspace, and VM Insights to continuously collect and visualize the health and performance metrics of the Linux Virtual Machine. Performance dashboards were created to monitor CPU, memory, network, and disk activity in real time. CPU load was intentionally generated on the VM using stress-ng to validate live metric collection and dashboard updates.

Step-by-step Execution and Evidence:

1. Azure Monitor and VM Insights were enabled. The virtual machine was configured to send performance metrics and guest operating system logs to

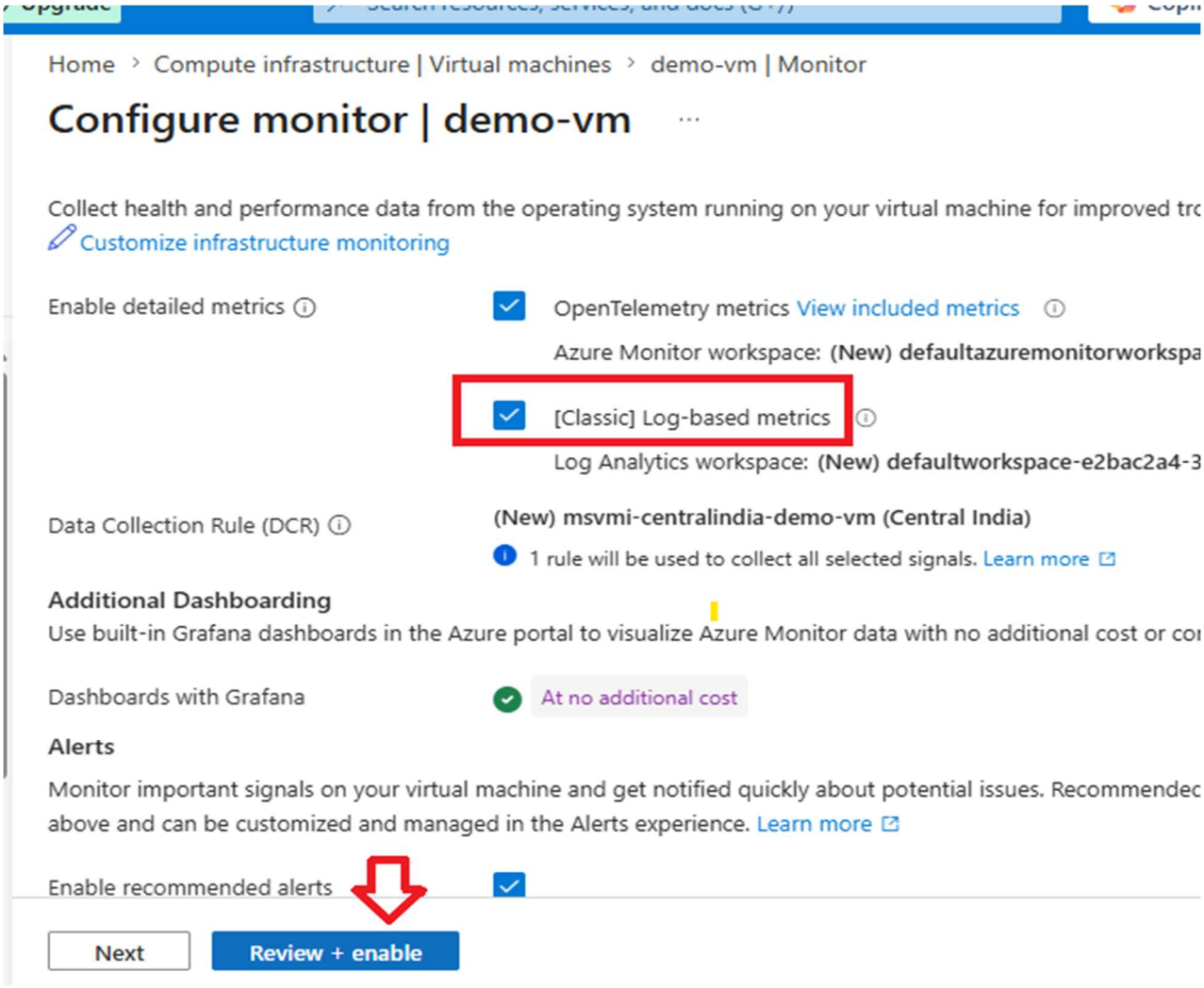

Figure 1: *Azure Monitor configuration showing* ***VM Insights***, ***Log Analytics Workspace***, *and data collection successfully enabled.*

2. CPU load was generated on the virtual machine using the stress-ng utility to simulate high processor utilization and validate real-time monitoring.

```
azureuser@demo-vm:~$ nproc
2
azureuser@demo-vm:~$ free -gh
               total        used        free      shared  buff/cache   available
Mem:           3.8Gi       605Mi       1.9Gi       4.8Mi       1.6Gi       3.2Gi
Swap:             0B          0B          0B
azureuser@demo-vm:~$ for i in {1.10000};do stress-ng --cpu 2 --timeout 240s; done &
[1] 51313
azureuser@demo-vm:~$ stress-ng: info:  [51314] setting to a 4 mins, 0 secs run per stressor
stress-ng: info:  [51314] dispatching hogs: 2 cpu
```

Figure 2: *Terminal showing the execution of the* ***stress-ng*** *command to generate CPU load on the Linux VM*

3. Azure Monitor Metrics were configured to collect performance data from the virtual machine. The Percentage CPU metric was selected to display processor utilization over time using a line chart.

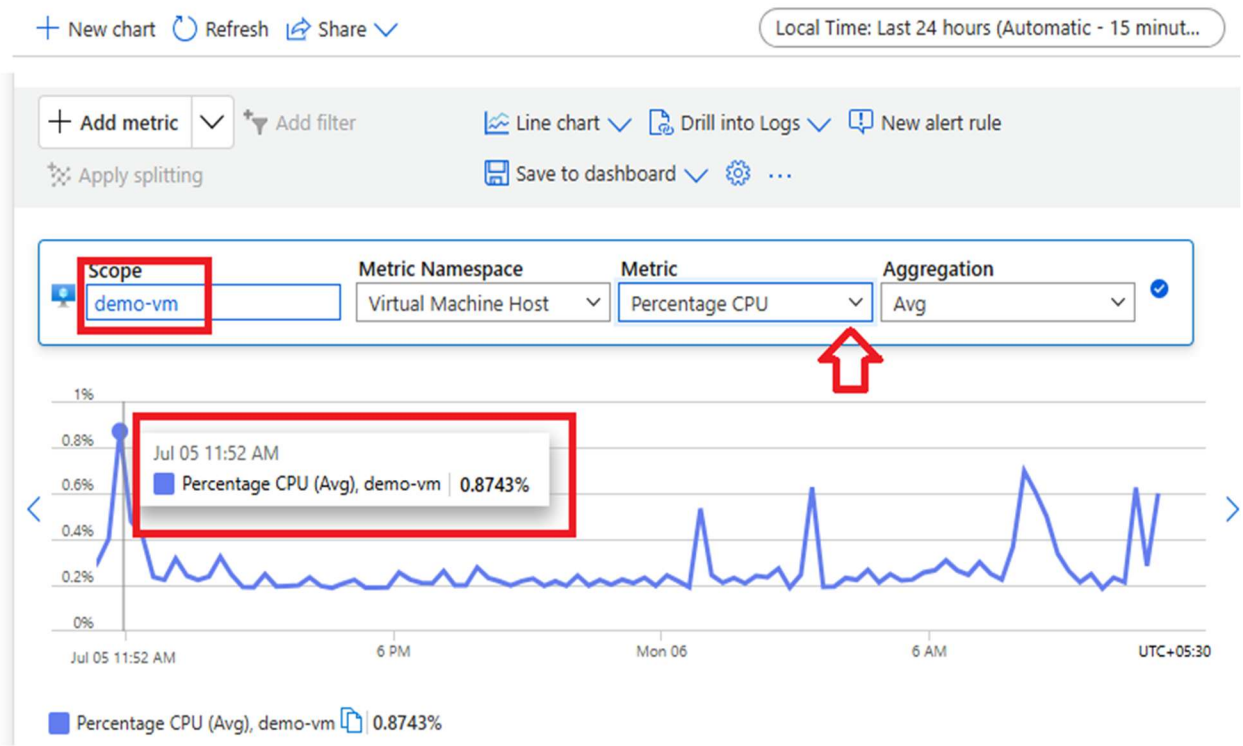

Figure 3: *Azure Monitor Metrics page displaying the* ***Percentage CPU*** *metric for the virtual machine using a line chart.*

4. A custom Azure Monitor Dashboard with Grafana was created to provide centralized visualization of the virtual machine's health and performance metrics.

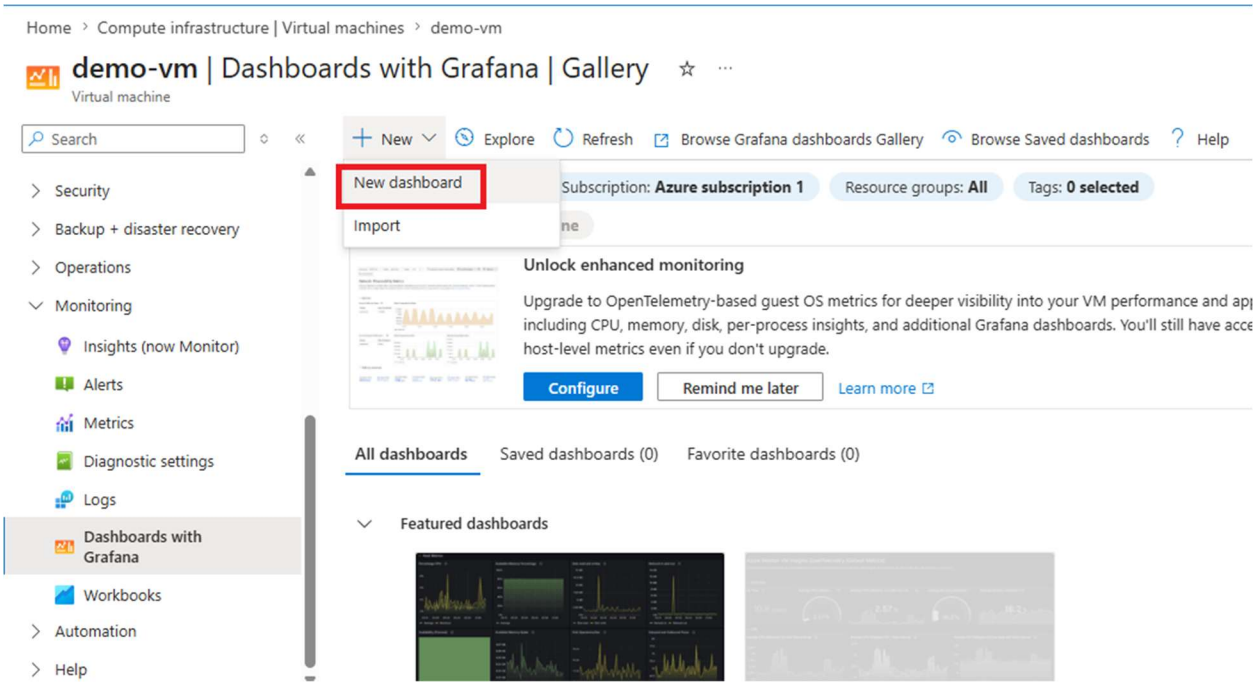

Figure 4: *Creating a new* ***Azure Monitor Dashboard with Grafana*** *from the Azure Portal.*

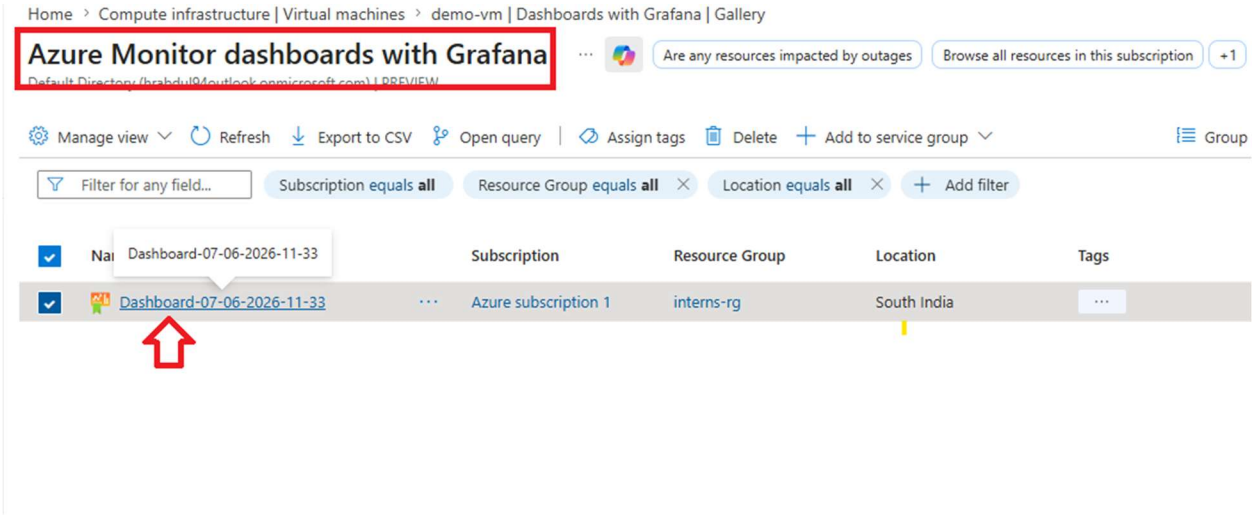

Figure 5: *Azure Monitor Dashboards gallery displaying the newly created dashboard.*

5. Dashboard panels were configured to display real-time performance statistics collected from Azure Monitor and Log Analytics Workspace.

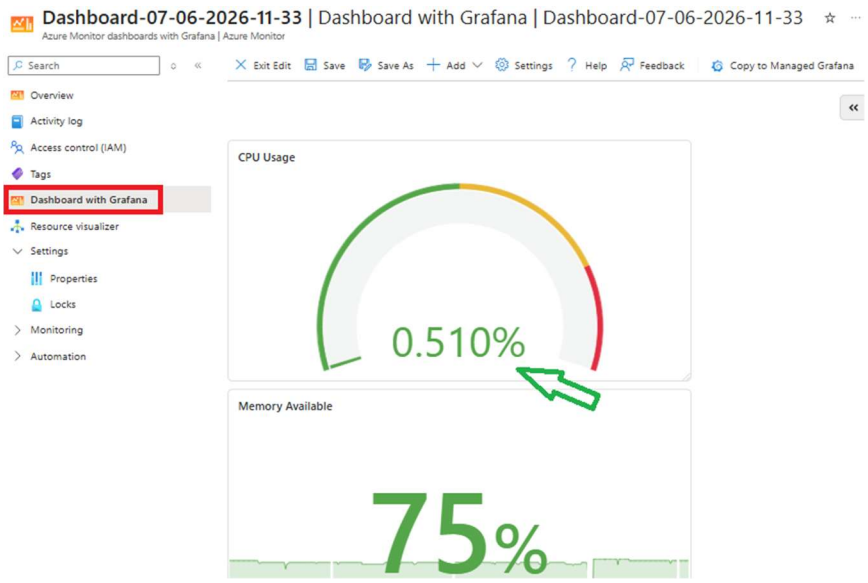

Figure 6: *Azure Monitor Dashboard displaying CPU and Memory monitoring panels.*

6. Monitoring validation was performed by generating sustained CPU utilization on the Linux VM. The dashboard reflected the increased processor usage in real time, confirming that Azure Monitor, VM Insights, and Log Analytics Workspace were successfully collecting and visualizing performance metrics.

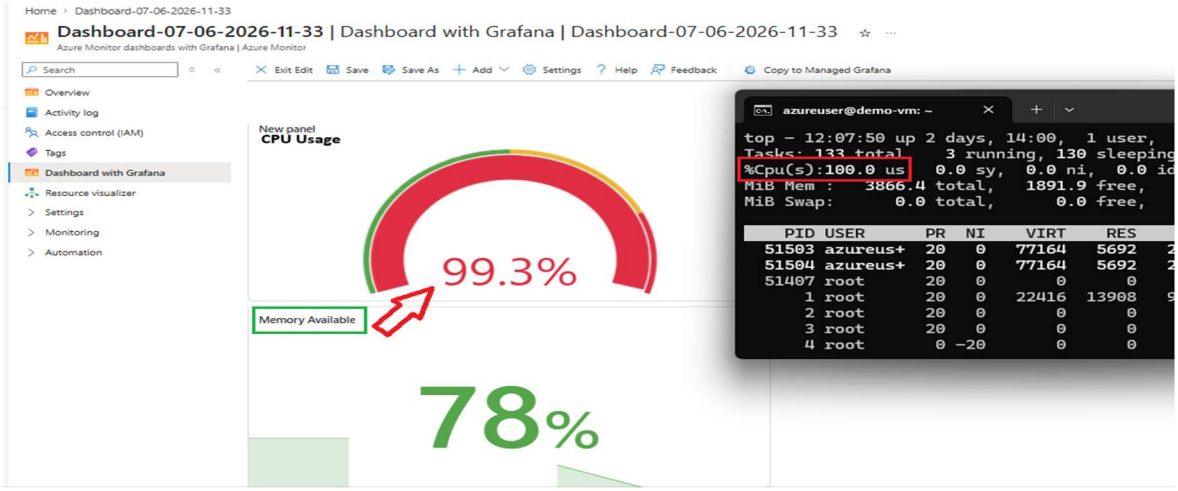

Figure 7: *CPU utilization reaching approximately **99%** on the Azure Monitor Dashboard while the Linux terminal simultaneously shows **100% CPU usage** generated by the **stress-ng** workload, validating successful real-time monitoring.*

Task 5: Microsoft Sentinel Integration

A Security Information and Event Management (SIEM) solution was implemented using Microsoft Sentinel to provide centralized security monitoring, log collection, threat detection, and incident management. A Log Analytics Workspace was configured to collect Linux Syslog events, while Microsoft Sentinel was connected to the workspace to analyze security logs. A scheduled analytics rule was created to detect multiple failed SSH login attempts and automatically generate security incidents.

Step-by-Step Execution and Evidence:

1. A Log Analytics Workspace was created to centrally collect monitoring and security logs generated by the Linux Virtual Machine.

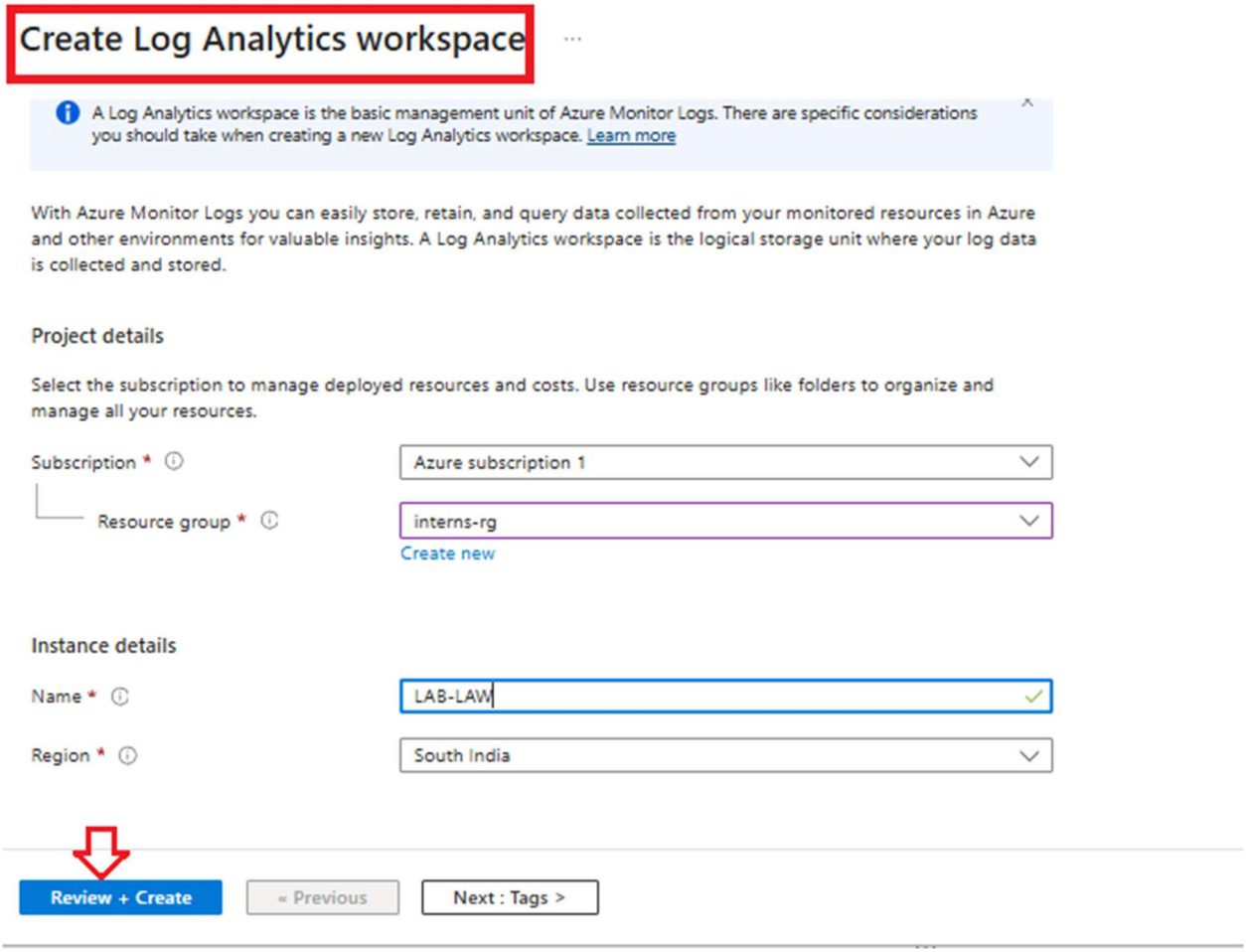

Figure 1: *Azure Portal showing the creation of the* ***Log Analytics Workspace (LAB-LAW)***

2. Azure Monitor Agent (AMA) was installed on the Linux Virtual Machine to forward operating system logs and performance data to the Log Analytics Workspace. The extension installation was verified using Azure CLI.

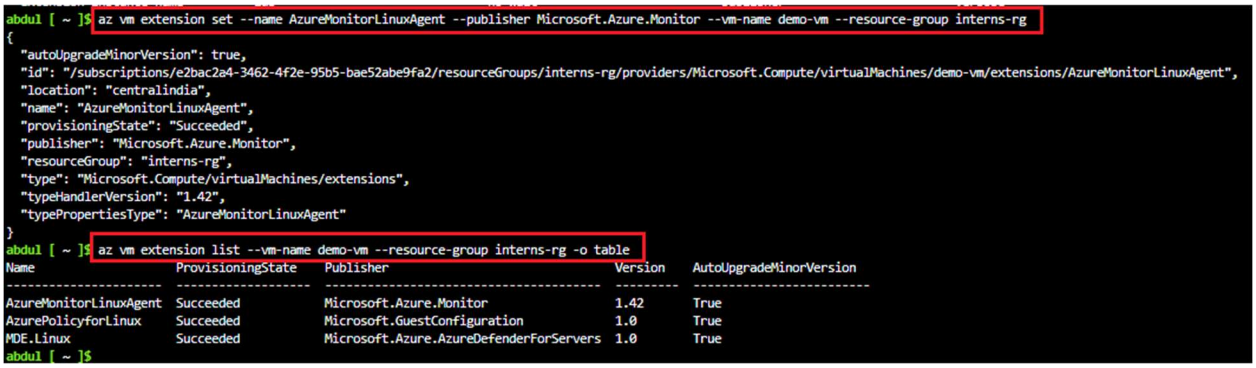

Figure 2: Azure CLI output confirming successful installation of the Azure Monitor Linux Agent and VM extensions.

3. A Data Collection Rule (DCR) was created to collect Linux Syslog events and send them to the Log Analytics Workspace for analysis.

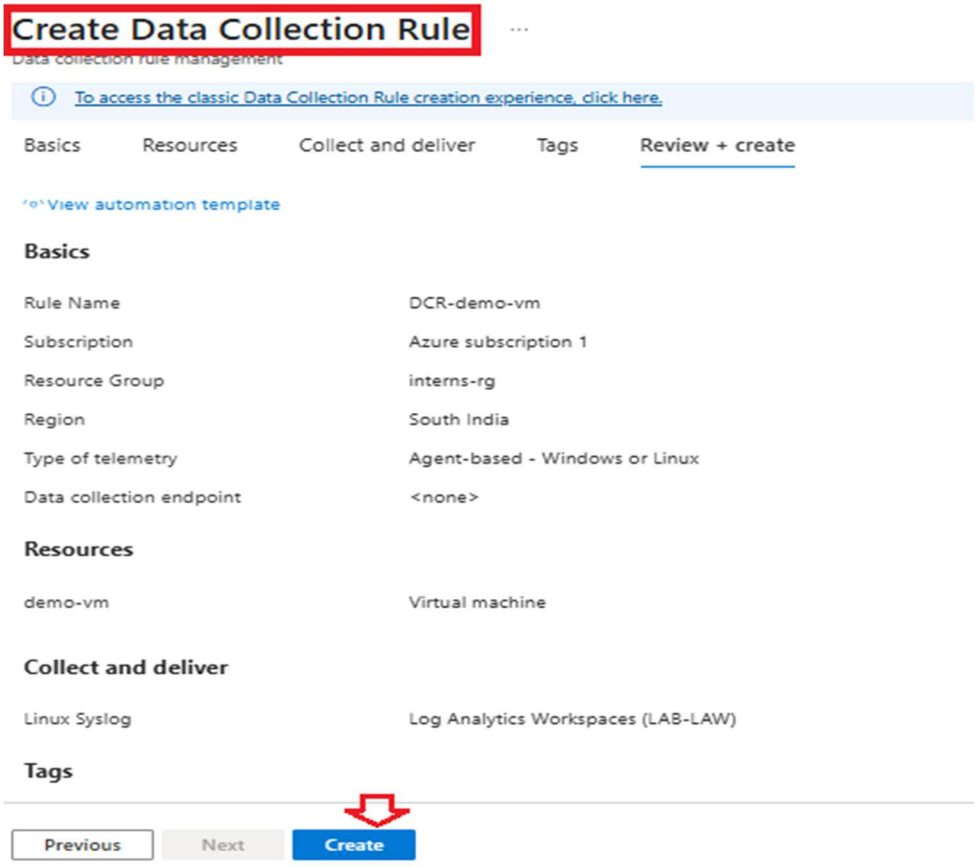

Figure 3: Review and creation of the Data Collection Rule (DCR) configured for Linux Syslog collection.

4. Syslog collection was validated by generating authentication events and reviewing the Linux authentication log (/var/log/auth.log). Failed login attempts and authentication failures were successfully recorded.

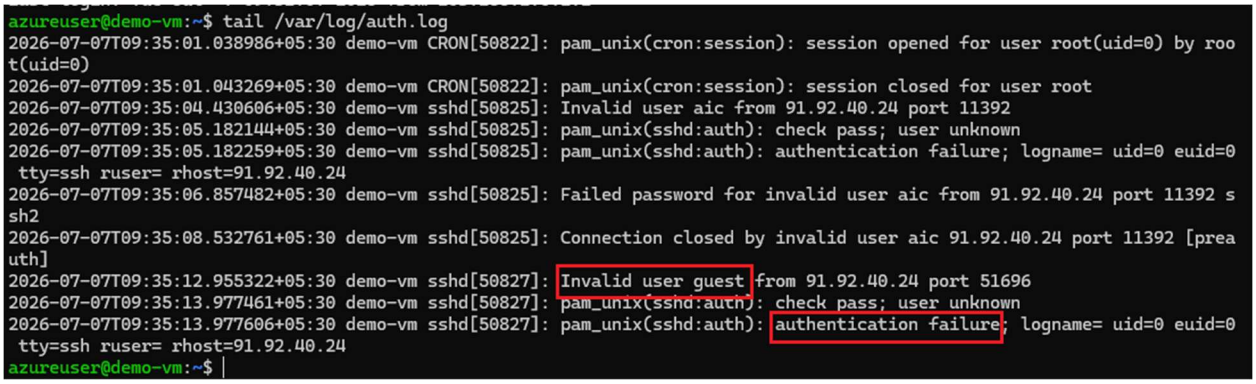

Figure 4: Linux authentication log displaying failed SSH login attempts and authentication failure events.

5. Log ingestion into Log Analytics Workspace was verified. A custom Syslog message was generated using the Linux logger command, and a Kusto Query Language (KQL) query confirmed that the event was successfully collected.

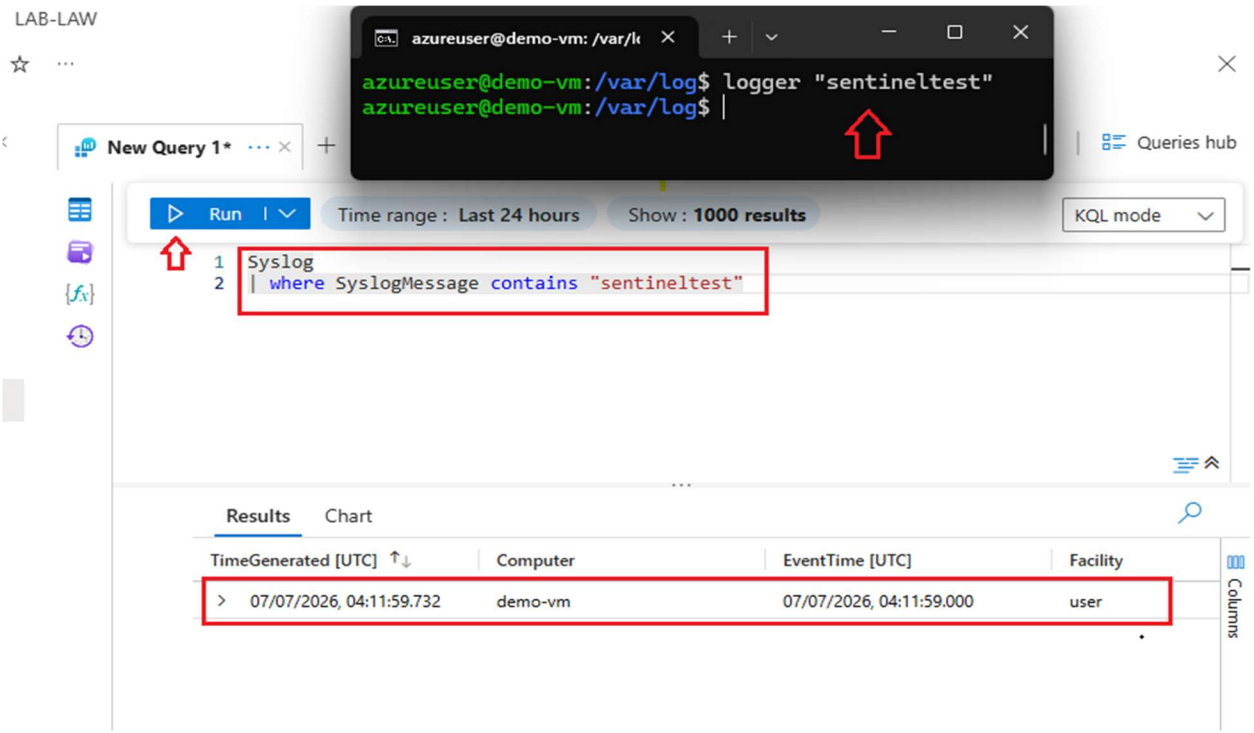

Figure 5: *KQL query successfully retrieving the custom Syslog event from the Log Analytics Workspace.*

6. VM connectivity with Log Analytics was confirmed by executing a Heartbeat query to verify that the Azure Monitor Agent was continuously reporting the health status of the virtual machine.

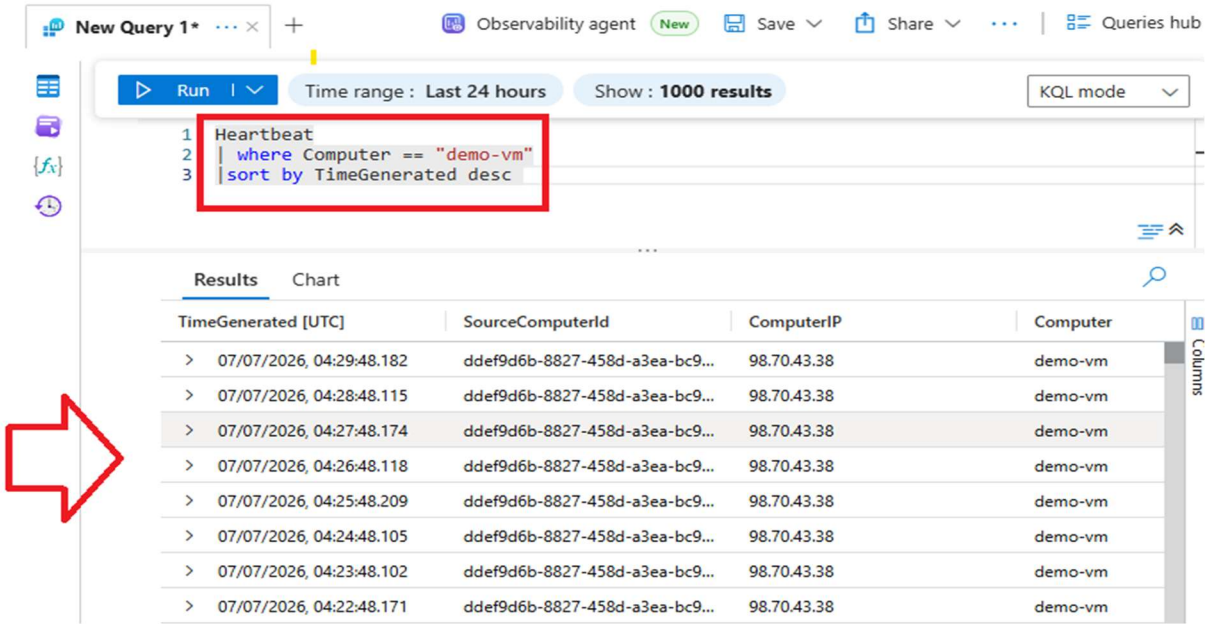

Figure 6: Heartbeat query results confirming that the virtual machine is actively sending monitoring data to Log Analytics.

7. Microsoft Sentinel was enabled on the Log Analytics Workspace, and a scheduled analytics rule was created to detect brute-force SSH attacks. The rule analyzes Syslog authentication events and triggers an alert whenever multiple failed SSH login attempts are detected within a defined time window.

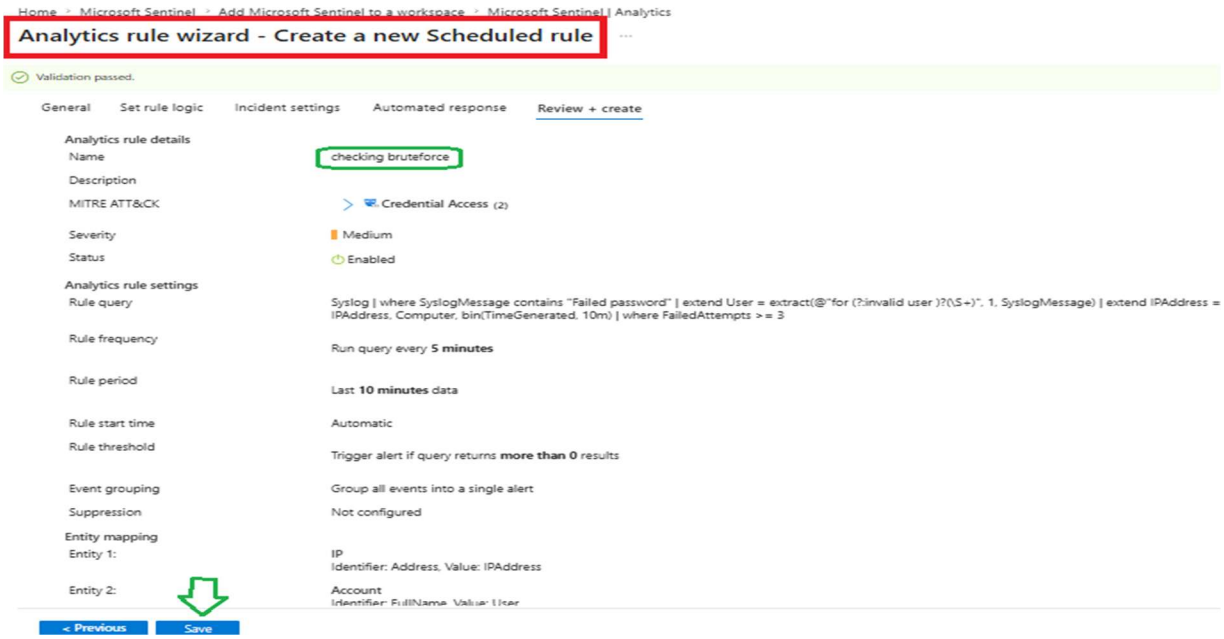

Figure 7: Analytics Rule Wizard showing the scheduled rule configuration for detecting SSH brute-force attempts.

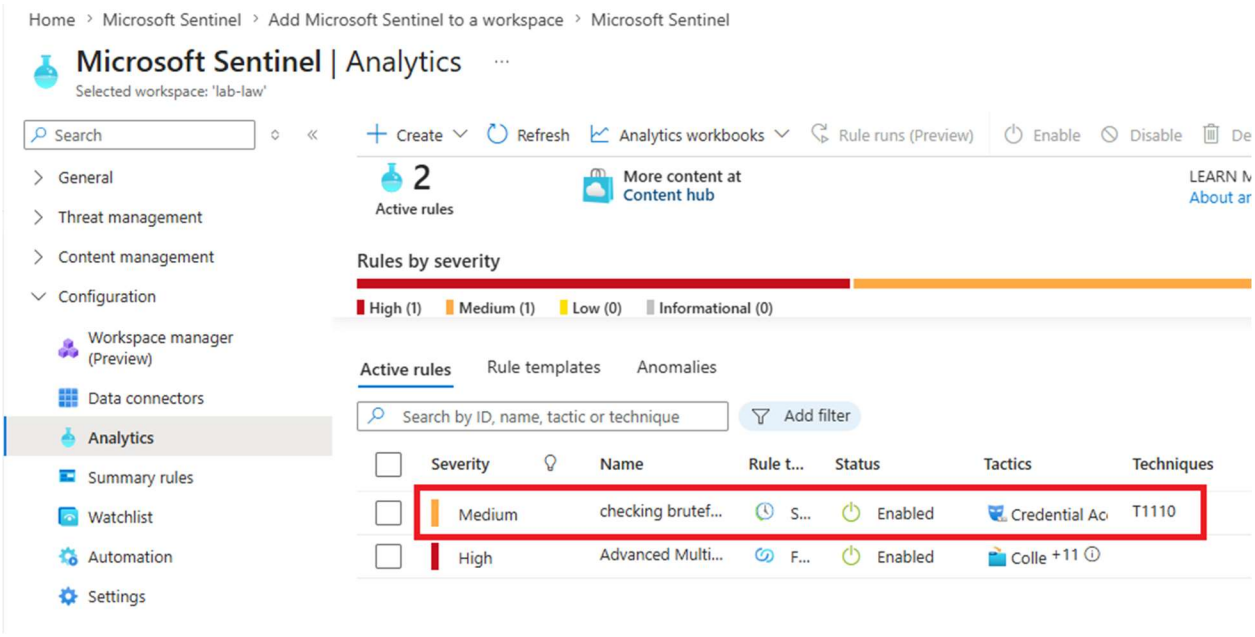

Figure 8: Microsoft Sentinel Analytics page displaying the enabled custom brute-force detection rule.

8. Multiple failed SSH login attempts were generated by intentionally connecting to the Linux virtual machine using invalid credentials. The failed authentication attempts were successfully recorded in the Linux authentication logs.

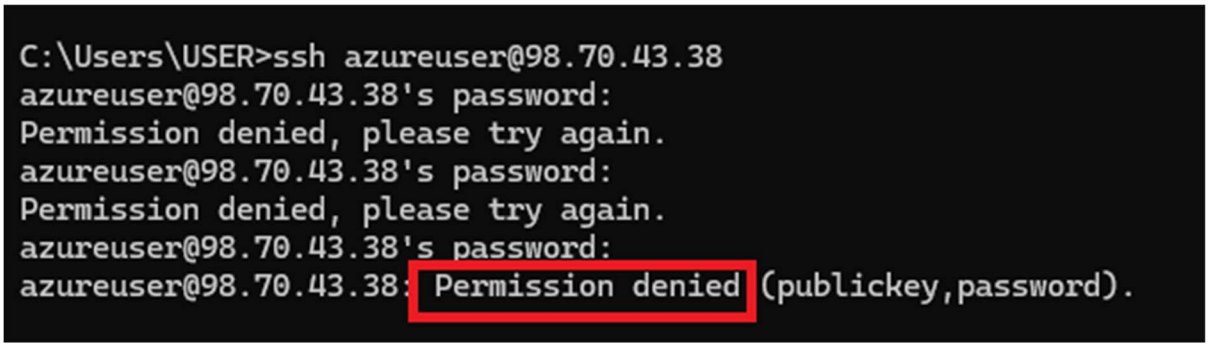

Figure 9: Windows SSH client displaying repeated Permission denied authentication failures.

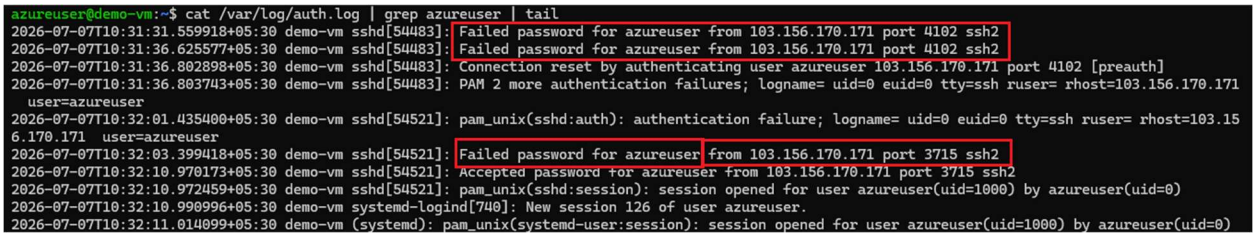

Figure 10: Linux authentication log showing multiple Failed password events generated by the test.

9. Microsoft Sentinel incident management was validated. The generated security alerts were forwarded to Microsoft Defender, where incidents were automatically created for the detected brute-force activity.

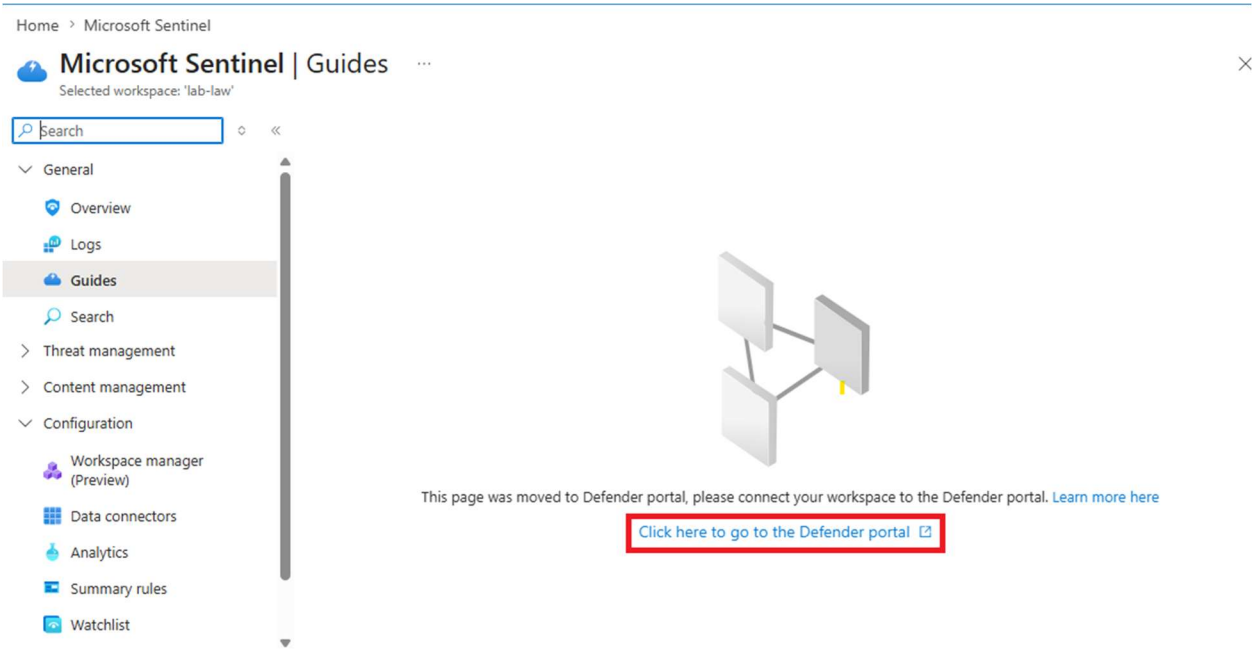

Figure 11: Microsoft Sentinel page showing the option to manage incidents through Microsoft Defender.

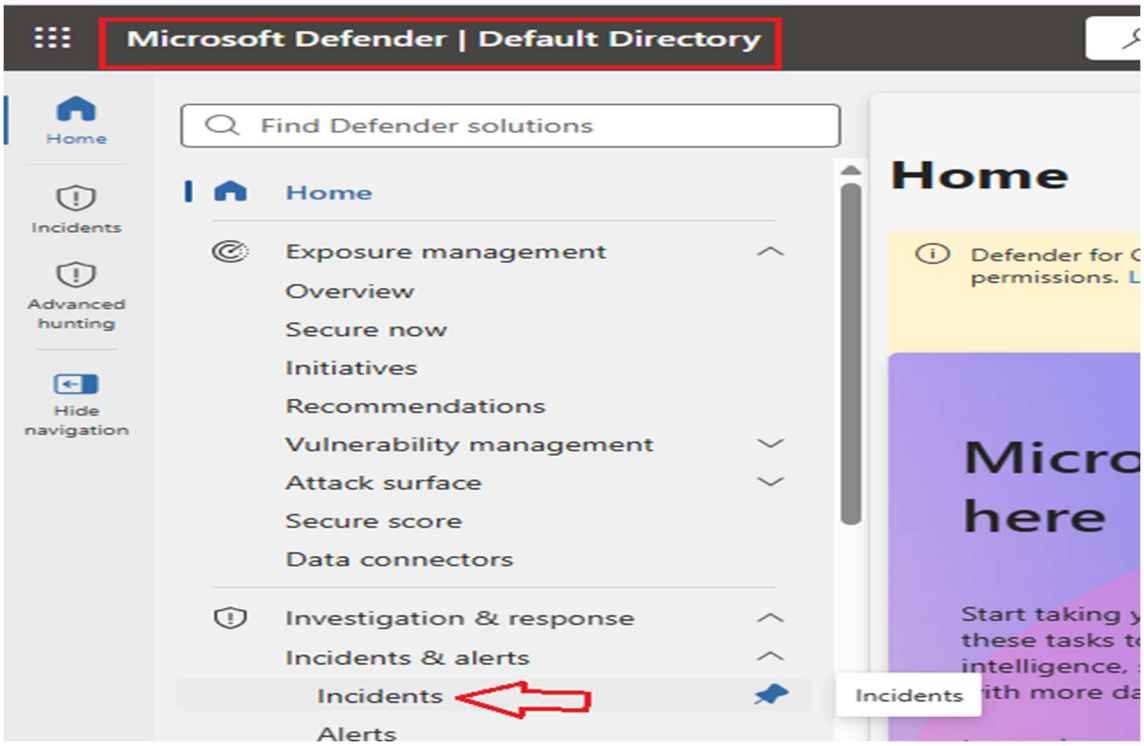

Figure 12: Microsoft Defender portal displaying the Incidents dashboard used for centralized security incident management.

10. Email notification was configured to notify administrators whenever the analytics rule generated a security incident.

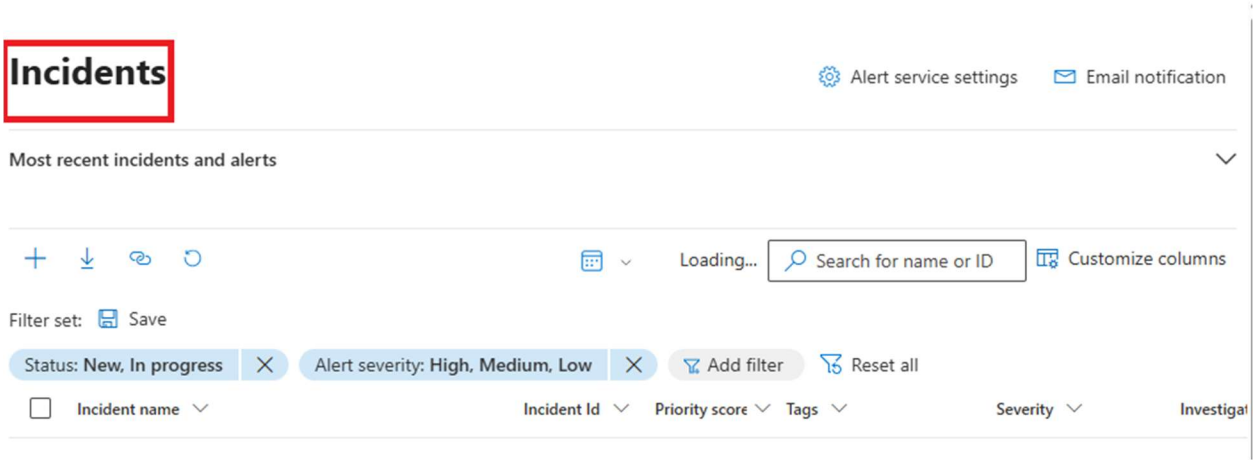

Figure 13: Notification rule configuration showing the administrator email address and alert settings.

11. The analytics rule successfully detected the brute-force activity and automatically generated security incidents in Microsoft Defender, confirming that Microsoft Sentinel was correctly collecting logs, executing KQL analytics, and producing security alerts.

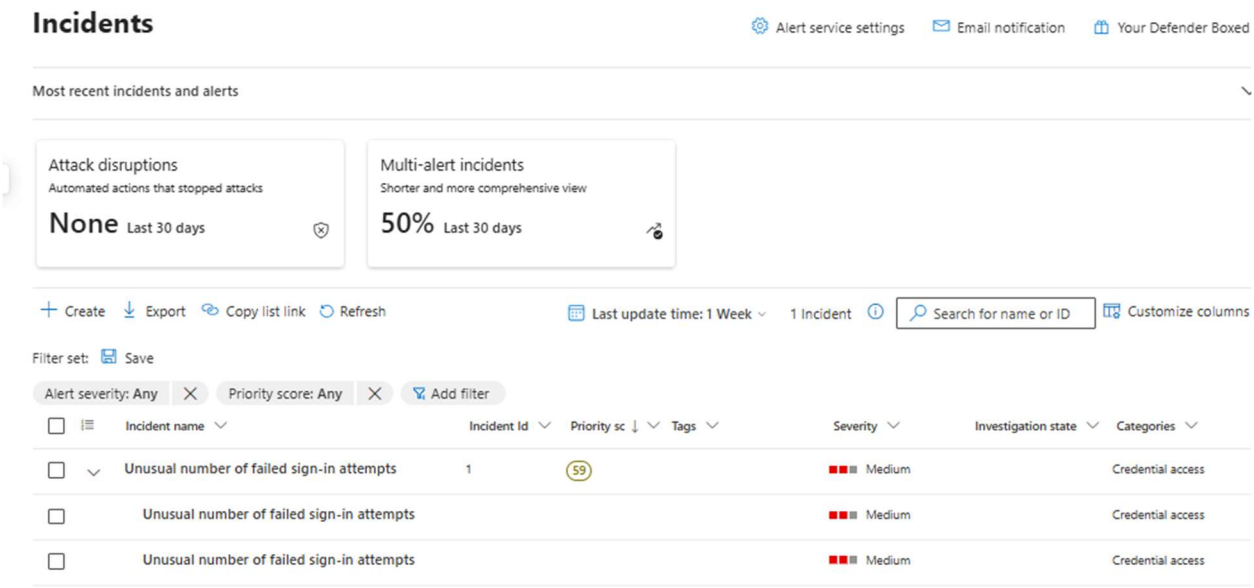

Figure 14: Microsoft Defender Incidents page displaying the generated "Unusual number of failed sign-in attempts" security incidents.

Task 6: Just-In-Time (JIT) Access

Just-In-Time (JIT) Virtual Machine Access was implemented using Microsoft Defender for Cloud to minimize the exposure of management and application ports. JIT keeps inbound ports closed by default and temporarily opens them only after an approved access request. In this implementation, Port 80 (HTTP) was protected using JIT access, allowing temporary access only from the administrator's IP address. After the approved time period expired, the port was automatically closed, restoring the secure state of the virtual machine.

Step-by-Step Execution and Evidence:

1. A temporary HTTP service was deployed on the Linux Virtual Machine using Python's built-in HTTP server. An initial attempt to access the web service from a browser failed because Port 80 was not publicly accessible.

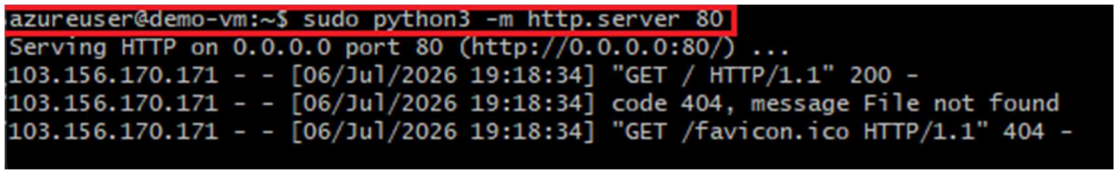

Figure 1: Linux terminal showing the Python HTTP server listening on Port 80.

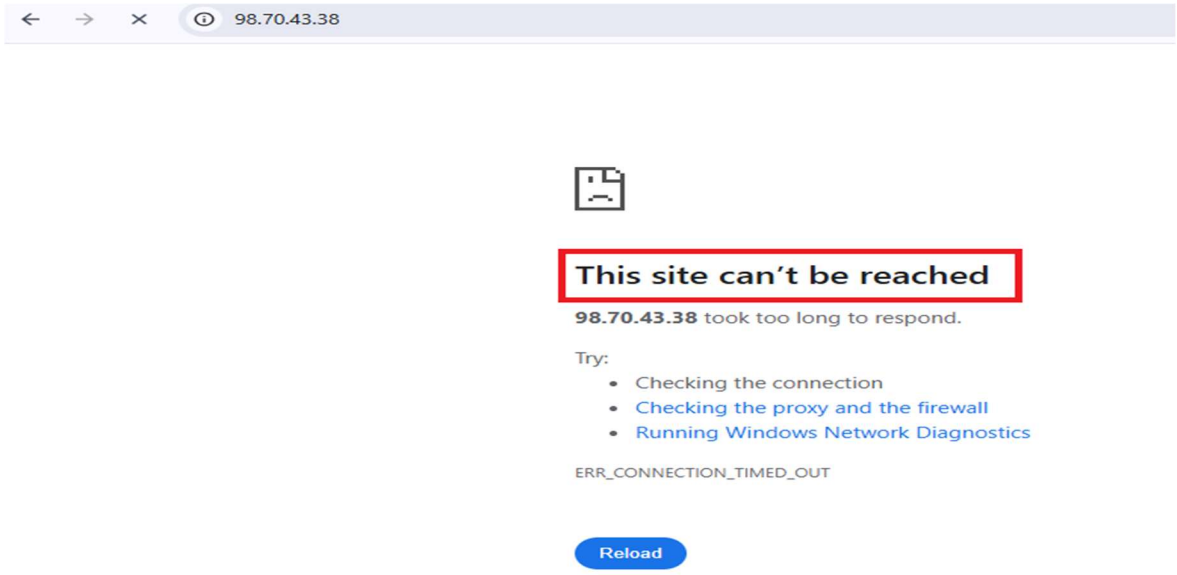

Figure 2: Web browser displaying "This site can't be reached", confirming that inbound access to Port 80 was blocked before enabling JIT access.

2. Microsoft Defender for Cloud protection was enabled for the Azure subscription by activating the Defender for Servers plan, allowing advanced security features such as Just-In-Time VM Access.

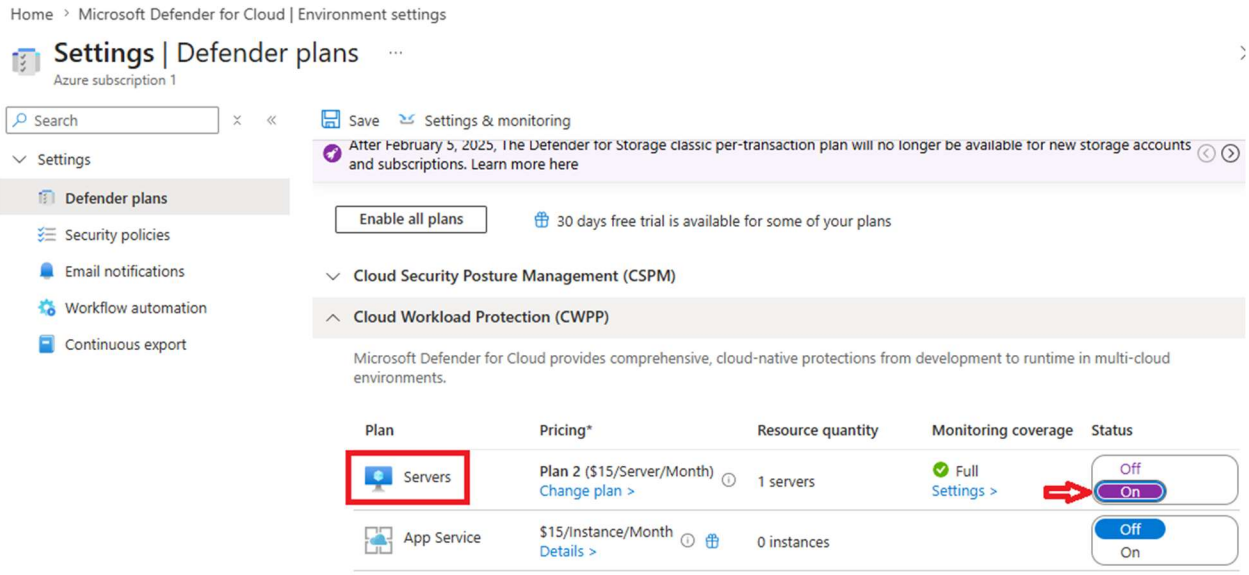

Figure 3: Microsoft Defender for Cloud showing the Defender for Servers plan enabled.

3. Just-In-Time (JIT) VM Access was enabled for the Linux virtual machine from the Azure Portal.

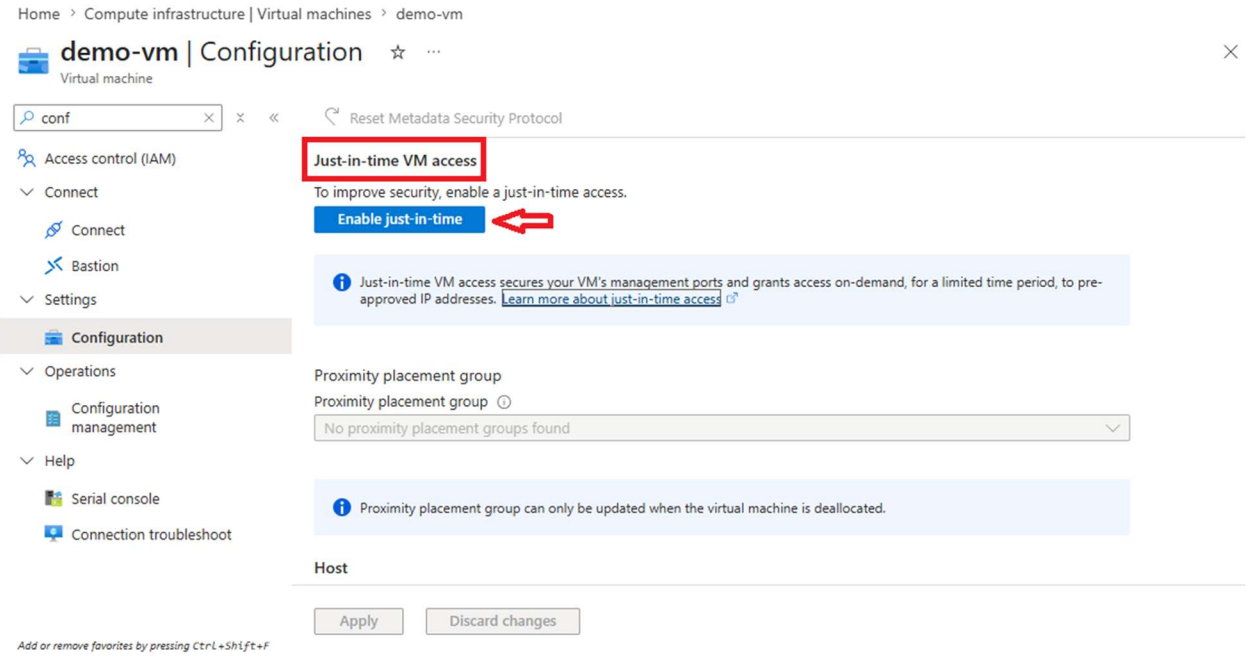

Figure 4: Virtual Machine configuration page showing Just-In-Time VM Access enabled.

4. Port 80 was configured for JIT protection. A new JIT policy was created to protect HTTP traffic, allowing access only upon approved requests. The policy was configured to permit access from the requester's public IP address for a limited duration.

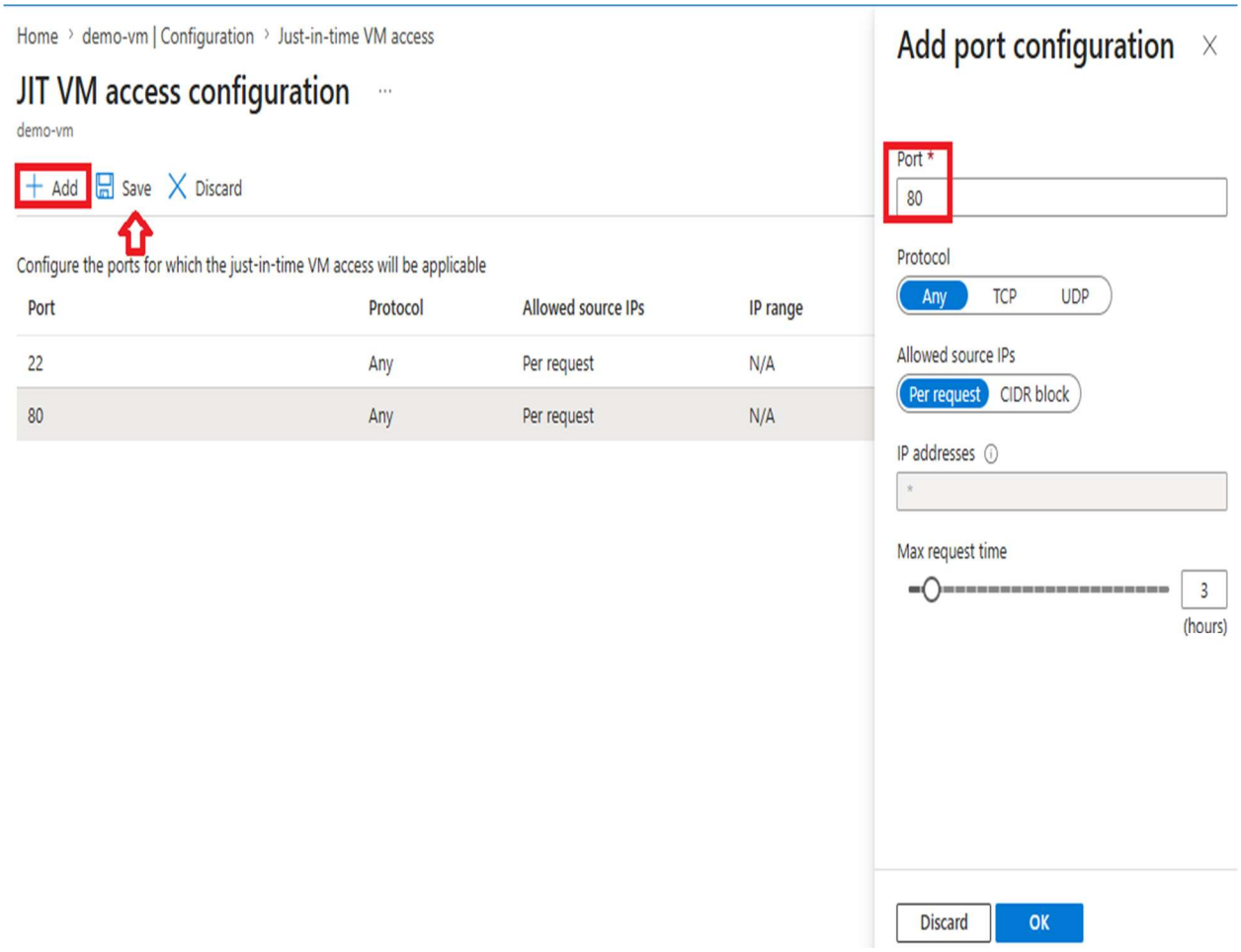

Figure 5: JIT VM Access configuration showing Port 80 added to the protected ports list with Per Request access control.

5. Temporary access to Port 80 was requested. The administrator selected My IP as the allowed source address and submitted a request to temporarily open Port 80.

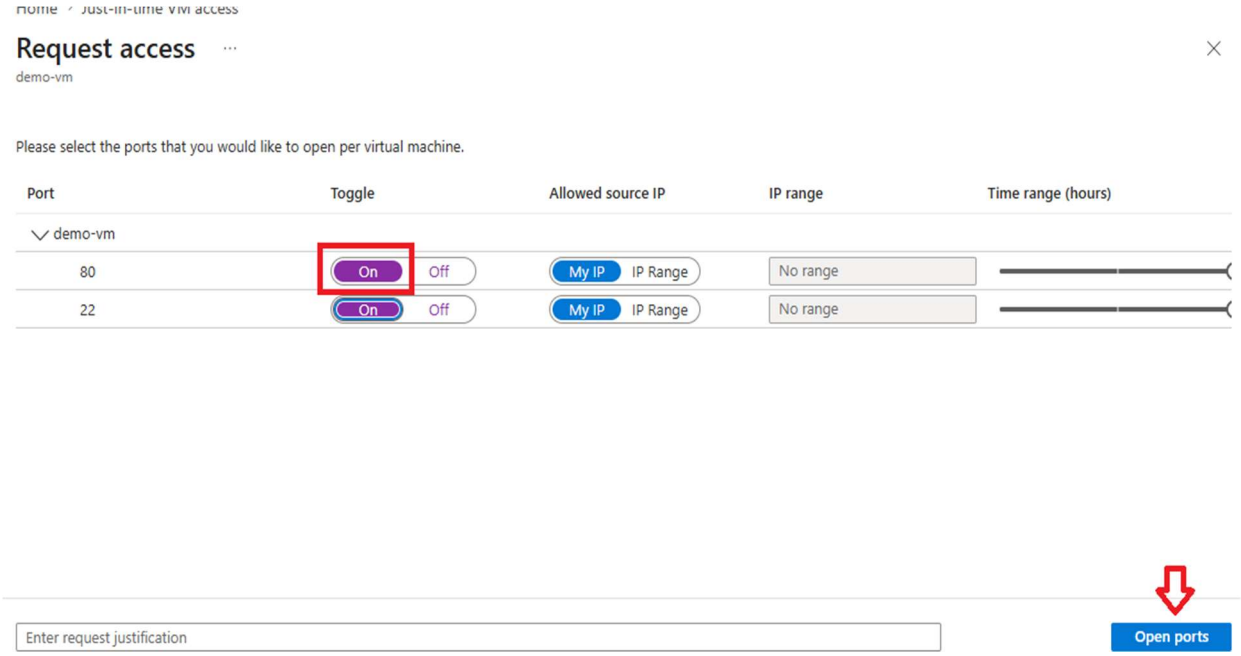

Figure 6: JIT Access Request page showing temporary access requested for Port 80 and approved by Microsoft Defender for Cloud.

6. After approval, the Network Security Group was automatically updated by JIT to temporarily allow inbound traffic on Port 80 from the approved source IP address. The previously inaccessible web application became reachable from the browser, confirming successful temporary access.

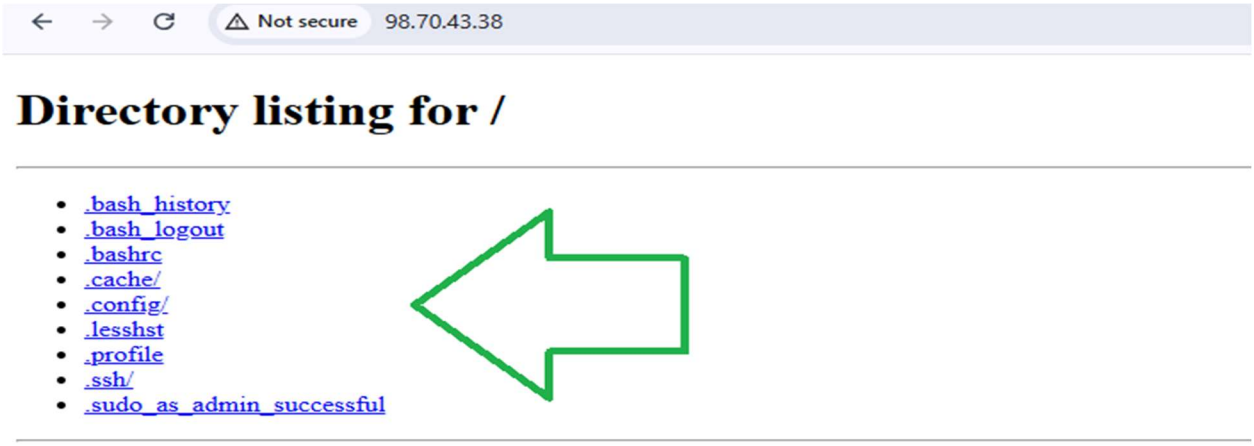

Figure 7: Web browser successfully accessing the Python HTTP server after JIT approval, displaying the default directory listing page.

Task 7: Vulnerability Assessment

A basic vulnerability assessment was performed using Nmap from a Kali Linux system to identify open ports, enumerate running services, and evaluate the exposed attack surface of the target host. Multiple Nmap scan techniques were used, including port scanning, service version detection, operating system detection, default script scanning, and result export. The findings provide an overview of accessible services and highlight potential security risks that should be addressed.

Step-by-Step Execution and Evidence:

1. Target system information was identified before performing the assessment. The private IP address of the Window local Machine.

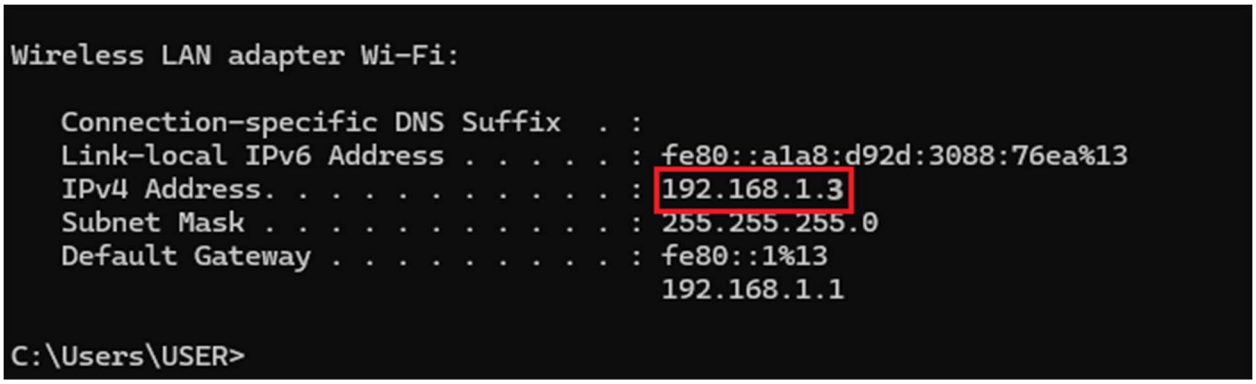

Figure 1: *Azure Virtual Machine overview displaying the target system's public IP address.*

2. An initial TCP port scan was performed using Nmap to discover open ports on the target system.

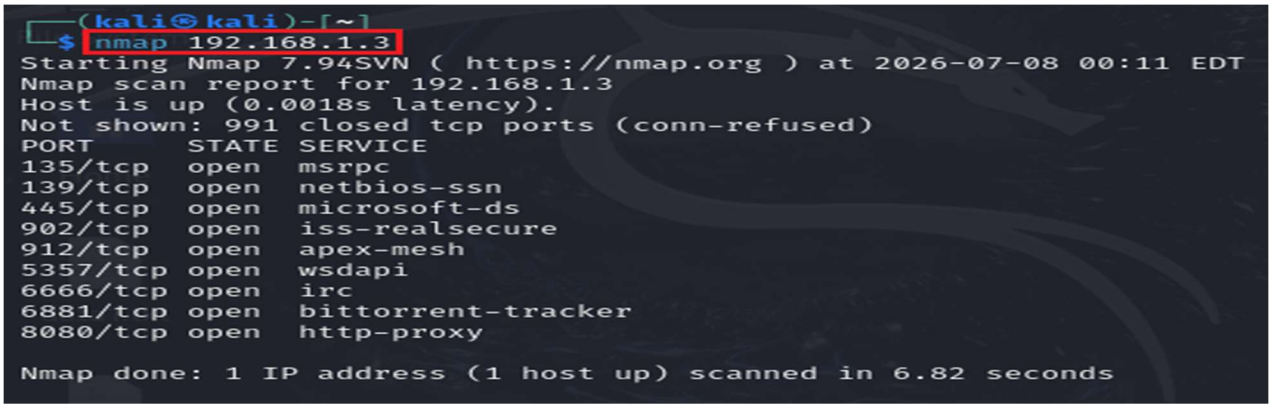

Figure 2: Nmap scan results showing the list of open TCP ports discovered on the target host.

3. Service version detection was executed to identify the applications and services running on the discovered ports.

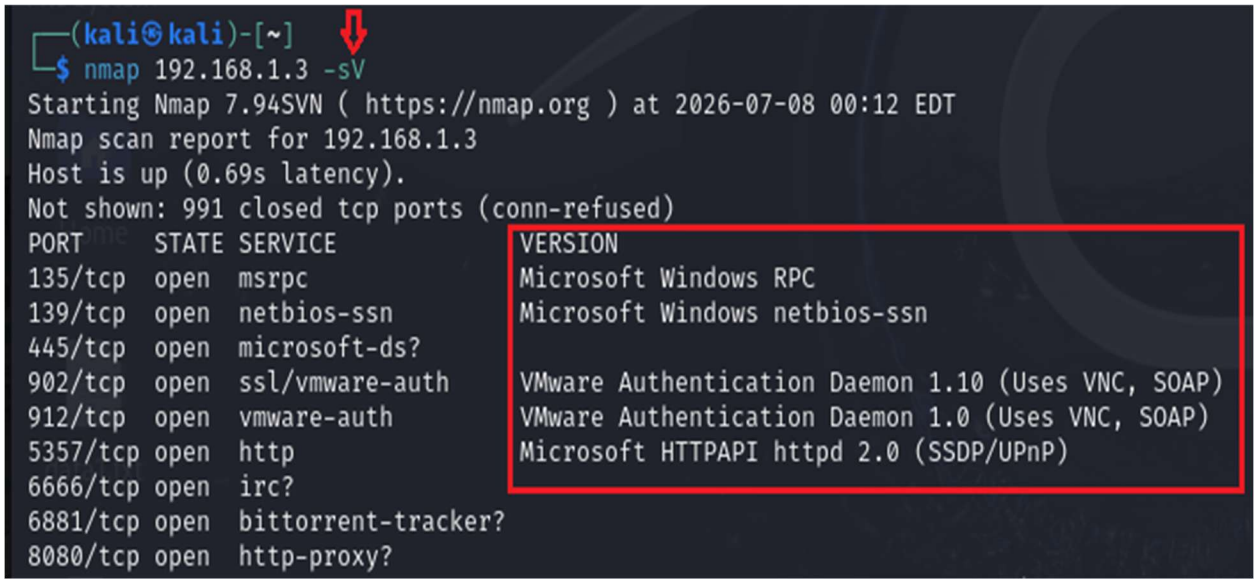

Figure 3: Nmap service enumeration displaying detected services and software versions.

4. Operating system detection was attempted to identify the operating system of the target machine.

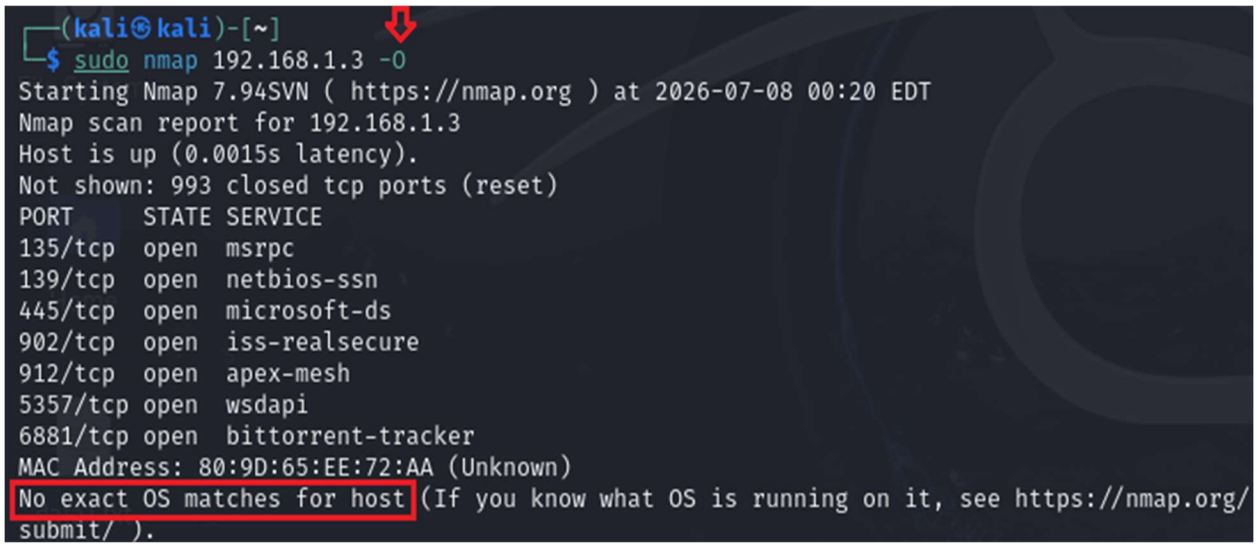

Figure 4: Operating system detection results showing no exact OS match.

5. An aggressive service scan was performed to gather additional information including service banners, HTTP headers, and application details.

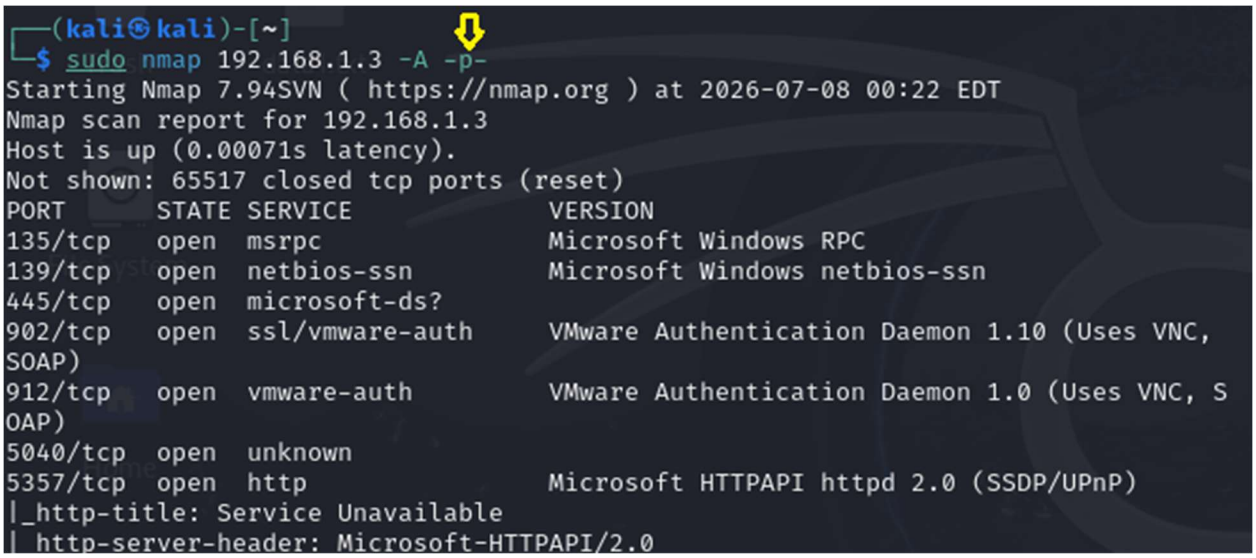

Figure 5: Aggressive Nmap scan displaying detailed service information and HTTP banner enumeration.

6. Extended port analysis was performed to identify additional high-numbered ports and exposed services that may increase the attack surface.

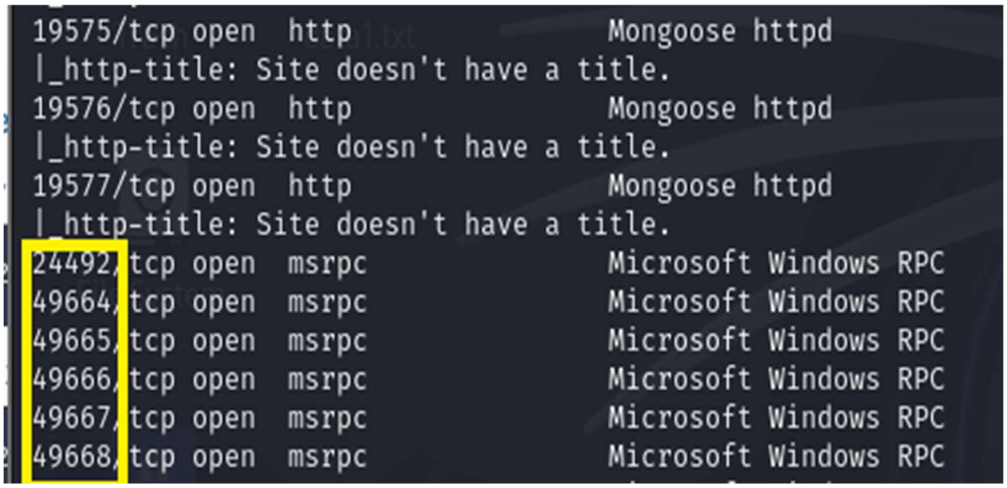

Figure 6: Nmap output showing additional open ports and services discovered during the assessment.

7. The scan results were exported for documentation and future analysis using Nmap's normal output format.

Figure 7: Command used to export the Nmap scan results into a text file.

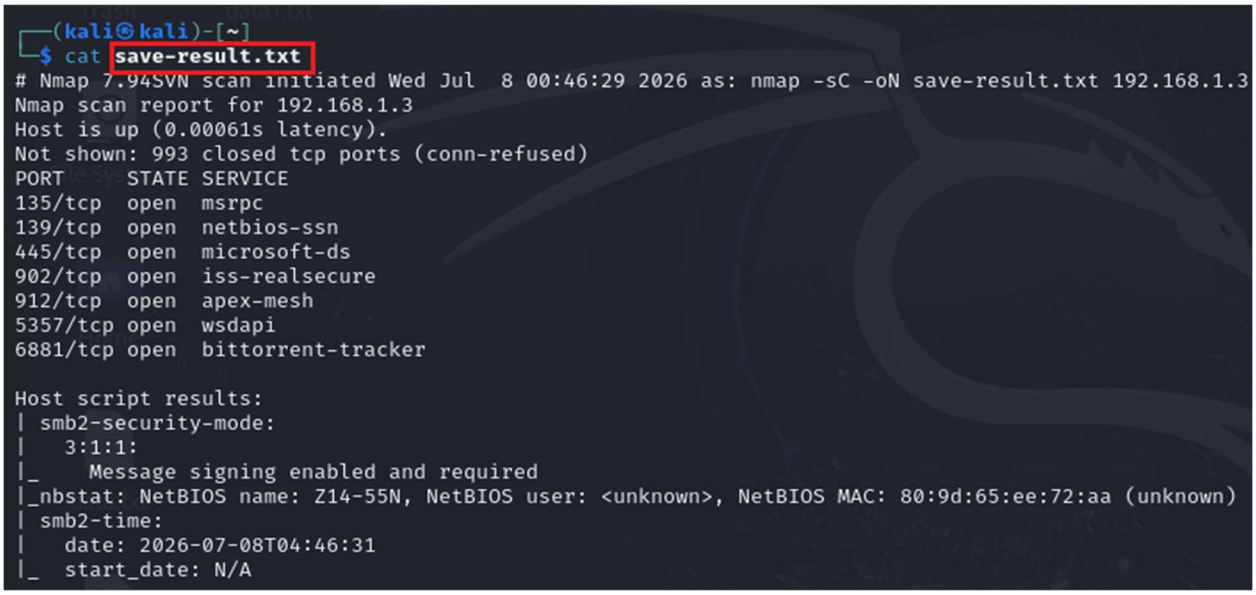

Figure 8: Contents of the exported save-result.txt file containing the complete scan report.

Findings summary

Category	Findings
Open Ports	Multiple TCP ports were identified as open on the target host.
Running Services	Microsoft Windows RPC, NetBIOS, VMware Authentication Service, HTTP, and other network services were detected.
Service Enumeration	Service versions and HTTP banners were successfully identified for several exposed services.
Operating System Detection	OS fingerprinting was attempted; however, no exact operating system match was identified.
Report Generation	Scan results were successfully exported for documentation and future analysis.

Task 8: Security Assessment Report

1. Risks Identified

Before hardening, the environment exhibited a number of common cloud misconfiguration risks. These were identified during the baseline assessment and the Nmap vulnerability scan, and are summarized below.

1.1 Open Services and Public Exposure

- **Standard SSH (port 22) was reachable from the public internet on the Linux VM, making it a target for automated scanning and dictionary/brute-force attacks.**
- **A temporary HTTP service on port 80 was exposed with no restriction, meaning any inbound connection could reach the web service until Just-In-Time access was configured.**
- **The Nmap scan identified multiple open TCP ports on the target host, including services such as Microsoft Windows RPC, NetBIOS, and a VMware Authentication Service banner, indicating a broader attack surface than required for the VM's function.**

1.2 Weak Configurations

- **Default NSG rules permitted inbound traffic on well-known management ports without IP restriction, increasing exposure to credential-stuffing and brute-force attempts.**
- **Operating system fingerprinting during the Nmap scan returned no exact OS match, which on its own is not a vulnerability, but reflects that service banners and responses had not been reviewed for information leakage.**
- **Prior to hardening, there was no time-bound restriction on administrative ports; once opened, ports such as SSH remained accessible indefinitely rather than only when needed.**

1.3 Access Control Issues

- **Without RBAC segregation, a single administrative identity could potentially manage both compute and storage resources, increasing the blast radius if that identity were compromised.**
- **Storage accounts without delegated, scoped access controls force clients to either use long-lived master keys (high risk if leaked) or be granted broad directory-level permissions.**

2. Security Controls Implemented

The following controls were implemented and validated with hands-on testing to mitigate the risks identified in Section 2.

Control	Description	Risk Mitigated
RBAC (Role-Based Access Control)	Two dedicated identities (infrauser, storageuser) were created in Microsoft Entra ID and assigned scoped roles (VM/Network Contributor, Storage Account Contributor) at the resource group level.	Excessive privilege; single identity compromise leading to full tenant takeover.
NSG Rules (Network Security Group)	Custom inbound rules were defined to explicitly allow only required ports and block all other unsolicited inbound traffic to the VM.	Unrestricted inbound access; automated scanning and exploitation of unused ports.
Custom SSH Port (2244)	Default SSH port 22 was disabled; a custom rule was created to allow administrative access only on port 2244.	Mass automated brute-force attacks that specifically target the default SSH port.
Private Blob Storage	The 'folder1' container was configured with public access set to Private, so anonymous requests return 403/404 responses.	Anonymous internet access to business files and data leakage via indexed public URLs.
SAS Token Access	Read-only, time-bound Shared Access Signature tokens were generated to grant delegated access to specific blobs without exposing storage account keys.	Permanent exposure via long-lived shared links or leaked master keys.
IP Restrictions	Just-In-Time access rules and SAS parameters restrict connections to specific approved source IP addresses.	Access from unauthorized or unknown network locations.
Azure Monitor	VM Insights and a Log Analytics Workspace were configured to collect CPU, memory, disk, and network metrics, visualized on a Grafana-based dashboard.	Lack of visibility into resource health and abnormal performance patterns.

Control	Description	Risk Mitigated
Microsoft Sentinel	A SIEM/SOAR solution was connected to the Log Analytics Workspace with a scheduled analytics rule that detects repeated failed SSH logins and raises incidents in Microsoft Defender.	Undetected brute-force login attempts and delayed incident response.
JIT (Just-In-Time) Access	Microsoft Defender for Cloud was used to keep port 80 closed by default, opening it only for an approved source IP and a limited time window upon request.	Standing exposure of management/application ports outside of active use windows.

2.1 Verification Summary

- **SSH connections on port 22 were confirmed refused after hardening; connections on port 2244 succeeded.**
- **Cross-account testing confirmed infrauser was denied access to storage resources and storageuser was denied access to VM resources.**
- **Anonymous access to the blob container returned an access-denied/not-found error, while the generated SAS URL successfully rendered the file.**
- **A simulated CPU load using stress-ng was reflected in near real time on the Azure Monitor dashboard, confirming monitoring pipeline integrity.**
- **Repeated failed SSH logins were generated and successfully triggered a Microsoft Sentinel analytics rule, producing an incident in Microsoft Defender with an administrator email notification.**
- **JIT access to port 80 was requested, approved, and automatically revoked after the approved window, confirmed by browser accessibility testing before and after the approval window.**
- **An Nmap scan from an external Kali Linux host was used to independently verify the exposed attack surface of the hardened host, including port scanning, service/version detection, OS fingerprinting, and an aggressive scan with banner enumeration.**

3. Architecture Diagram

The diagram below illustrates the resource group boundary, virtual machine and NSG placement, RBAC identities, storage account and private container with SAS delegation, monitoring and Sentinel data flow, and JIT access control managed through Microsoft Defender for Cloud.

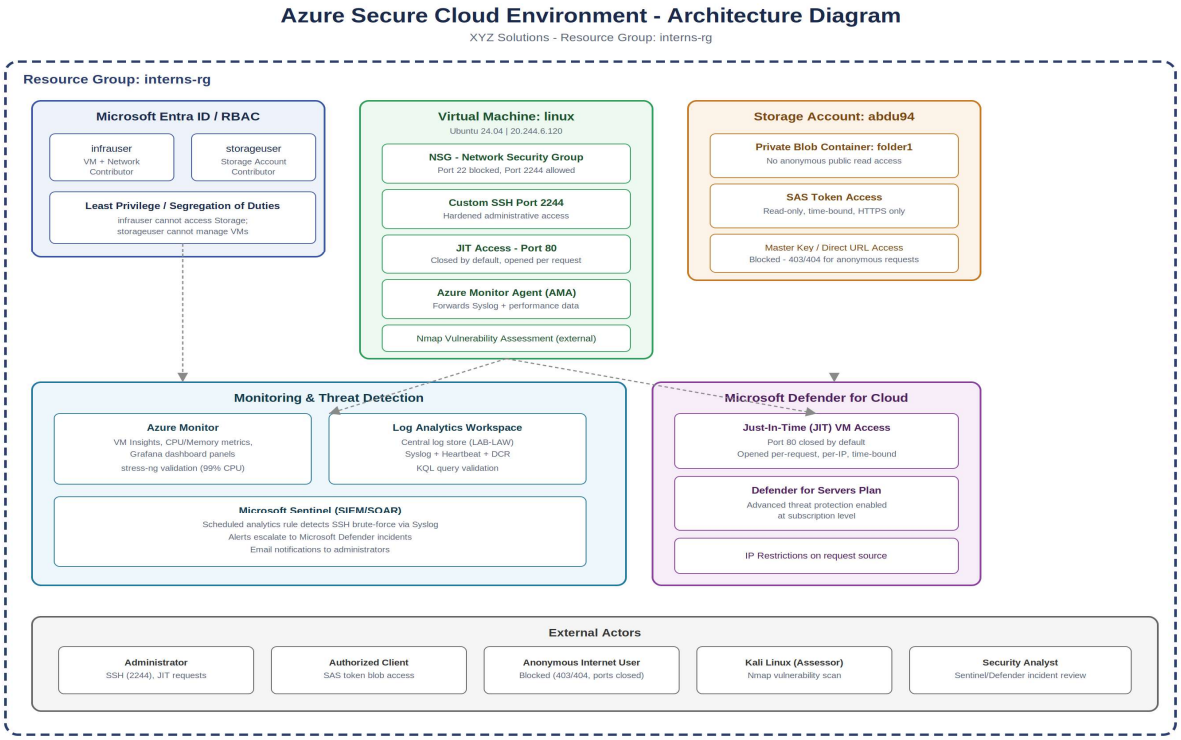

Figure A: Security architecture of the interns-rg environment, showing identity, compute, storage, monitoring, and JIT access control boundaries.

System Security Architecture Diagram

The cloud environment was structured inside an isolated Resource Group named 'interns-rg'. The diagram below illustrates the boundaries, network security group rules, identity roles, and storage container permission boundaries deployed:

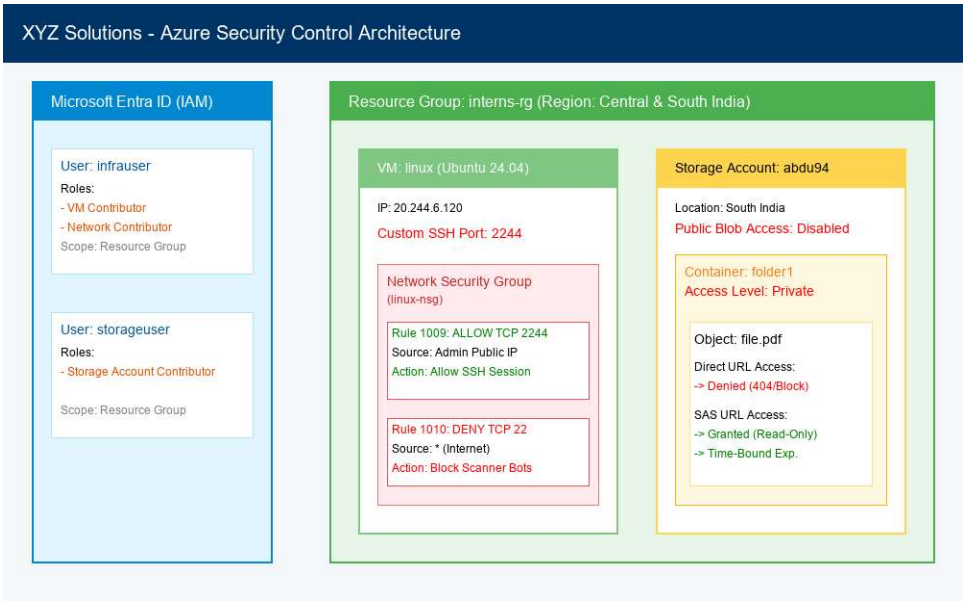

Figure A: System Architecture Diagram showing VM, network, IAM, and storage security perimeters.

4. Recommendations

The following actions are recommended to further mature the security posture of the XYZ Solutions environment beyond this baseline implementation.

4.1 Identity and Access

- **Enforce Multi-Factor Authentication (MFA) via Microsoft Entra ID Conditional Access for infrauser, storageuser, and all administrative accounts.**
- **Periodically review RBAC role assignments to confirm least-privilege scope remains appropriate as the environment grows.**

4.2 Network and Compute

- **Extend Just-In-Time access to cover the custom SSH port (2244) in addition to port 80, so administrative access is opened only on request.**
- **Reduce the attack surface identified by the Nmap scan by disabling or restricting unused services (e.g., RPC/NetBIOS) that are not required for the VM's workload.**
- **Consider Azure Bastion or a VPN gateway for administrative access instead of exposing any SSH port directly to the internet.**

4.3 Storage

- **Remove public network access paths entirely from the storage account and implement a Private Endpoint inside a dedicated Virtual Network (VNet).**
- **Adopt shorter SAS token lifetimes and stored access policies so tokens can be revoked centrally without rotating the storage account keys.**

4.4 Monitoring and Detection

- **Expand Microsoft Sentinel analytics rules beyond SSH brute-force detection to cover anomalous storage access, privilege escalation, and resource deletion events.**
- **Use the following KQL query as a starting template for brute-force detection tuning:**

```
Syslog | where ProcessName == "sshd" and SyslogMessage has "Failed password" | summarize FailedAttempts = count() by SourceIP = extract("from ([0-9.]+)", 1, SyslogMessage), Computer, TimeGenerated | where FailedAttempts > 5
```

- **Configure automated SOAR playbooks in Sentinel to automatically block a source IP or disable an account when brute-force incidents are confirmed.**

4.5 Governance

- **Document and schedule periodic vulnerability scans (e.g., recurring Nmap or Defender vulnerability assessments) rather than a single point-in-time scan.**
- **Maintain an updated architecture diagram and control inventory as new resources are added to the resource group.**

5. Final Deliverables

The following deliverables accompany this report as evidence of successful implementation and testing of each control:

- **VM Configuration Screenshots (provisioning, essentials, port hardening verification)**
- **RBAC Configuration Screenshots (Entra ID users, role assignments, access-denied validation)**
- **Storage Security Screenshots (private container, SAS token generation and validation)**
- **Azure Monitor Dashboard Screenshots (VM Insights, Grafana panels, stress-ng validation)**
- **Microsoft Sentinel Screenshots (analytics rule, incidents, Defender integration)**
- **JIT Access Screenshots (policy configuration, request/approval, before/after access validation)**
- **Nmap Scan Results (port scan, service detection, OS detection, exported report)**